

Le Langage SQL

Traitement des données

Le Langage de Manipulation de Données

L'ajout, la suppression et la modification de données

A decorative graphic on the left side of the slide consists of a vertical arrangement of blue and yellow hexagons of various sizes, some solid and some outlined, creating a pixelated or molecular-like structure.

Introduction

■ Les ordres SQL :

- INSERT
- UPDATE
- DELETE

■ Ces instructions SQL influent directement sur les données, contrairement à l'instruction SELECT



L'insertion de données (1)

■ La syntaxe générale

INSERT INTO nom_de_la_table_cible [(liste_des_colonnes_visées)] **VALUES**
(liste_des_valeurs)

[] optionnel dans la requête

■ 3 types d'insertion

- Insertion explicite
- Insertion multiple
- Insertion à base de sous requête **SELECT**

L'insertion de données (2)

■ Insertion simple explicite

On insère directement les valeurs dans les colonnes de la table,
!! insertion d'une seule ligne !!

Exemple :

*insertion d'une ligne dans la table T_MODE_PAIEMENT,
elle contient deux colonnes (code et libelle)*

```
INSERT INTO T_MODE_PAIEMENT (CODE, LIBELLE)  
VALUES('VIR','virement bancaire');
```

OU

```
INSERT INTO T_MODE_PAIEMENT  
VALUES('VIR','virement bancaire');
```

L'insertion de données (3)

■ Insertion multiple

On insère plusieurs lignes dans la table cible,

Exemple :

*insertion de trois lignes dans la table T_MODE_PAIEMENT,
elle contient deux colonnes (code et libelle)*

```
INSERT INTO T_MODE_PAIEMENT (CODE, LIBELLE)  
      VALUES      ('VIR','virement bancaire'),  
                   ('CB', 'Carte Bancaire'),  
                   ('CHQ','Chèque');
```

L'insertion de données (4)

■ Insertion partiellement explicite

Exemple de création de table

```
CREATE TABLE CONNEXION
```

```
(  
    C_USER    VARCHAR(128)          NOT NULL DEFAULT 'Administrateur',  
    DATE_HEURE TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO CONNEXION (C_USER, DATE_HEURE) VALUES('Dupont', DEFAULT);
```

```
INSERT INTO CONNEXION VALUES('Durant', DEFAULT);
```

```
INSERT INTO CONNEXION (C_USER) VALUES('Duchemol');
```

```
INSERT INTO CONNEXION VALUES ( DEFAULT, DEFAULT);
```

c_user	date_heure
Dupont	2015-02-05 11:18:22.654

c_user	date_heure
Dupont	2015-02-05 11:18:22.654
Durant	2015-02-05 11:19:37.743

c_user	date_heure
Dupont	2015-02-05 11:18:22.654
Durant	2015-02-05 11:19:37.743
Duchemol	2015-02-05 11:20:33.544

c_user	date_heure
Dupont	2015-02-05 11:18:22.654
Durant	2015-02-05 11:19:37.743
Duchemol	2015-02-05 11:20:33.544
Administrateur	2015-02-05 11:21:29.957

L'insertion de données (5)

■ Insertion à base de sous requêtes

-- sous requête qui retourne une seule colonne et une seule ligne

```
INSERT INTO CONNEXION
```

```
VALUES( (select NOM from CLIENT where CLI_ID=4), default);
```

-- sous requête qui retourne plus d'une ligne

```
INSERT INTO CLIENT (CLI_ID, CODE, NOM, PRENOM)
```

```
SELECT PRP_ID, PRP_CODE_TITRE, PRP_NOM, PRP_PRENOM
```

```
FROM T_PROSPECT ;
```

La suppression de données (1)

■ La syntaxe générale

DELETE FROM nom_table_cible
[WHERE condition]
[RETURNING *]

[] *optionnel dans la requête*

■ les types de suppression

- Suppression totale
- Suppression conditionnelle
- Suppression et sous requête

La suppression de données (2)

- **Suppression totale**

```
DELETE FROM PROSPECT;
```

- **Suppression conditionnelle**

```
DELETE FROM PROSPECT  
WHERE prenom LIKE '%A%' ;
```

- **Suppression et sous requête**

```
DELETE FROM CLIENT  
WHERE cli_id IN (SELECT prp_id FROM PROSPECT);
```

- **Avec retour des lignes supprimées**

```
DELETE FROM CLIENT  
WHERE cli_id IN (SELECT prp_id FROM PROSPECT)  
RETURNING *; -- * Pour toute les colonnes ou sinon lister les colonnes
```

La suppression de données avec la clause USING(3)

- La Clause USING permet de faire une pseudo jointure avec une autre table.

```
DELETE FROM table1  
USING table2  
WHERE condition  
RETURNING returning_columns;
```

- Exemple :

```
DELETE FROM circuit C  
USING pilote P  
WHERE C.idpilote = P.idpilote;
```

Ici on supprime les lignes de la table circuit quand il existe les pilotes dans la table pilote

La suppression de données avec la clause USING(3)

■ Autre exemple

```
DELETE FROM grandprix G
USING circuit C
WHERE G.id_circuit= C.id_circuit
and payscircuit = 'Autriche';
```

Ici on supprime les lignes de la table grandprix quand le circuit sur lequel il se déroule et en Autriche

La suppression de données (3)

■ Autre instruction SQL

TRUNCATE supprime rapidement toutes les lignes d'un ensemble de tables. Elle a le même effet qu'un DELETE sans condition, mais comme elle ne parcourt pas la table, elle est plus rapide.

■ **Syntaxe :** **TRUNCATE** nomtable

■ **Exemple :**

Vide la table personne : **TRUNCATE** personne ;

Vide la table personne et adresse : **TRUNCATE** personne, adresse ;

Vide les tables en cascade : **TRUNCATE** cereale **CASCADE**;

suppression en cascade de toutes les tables qui référencent cereale via des contraintes de clés étrangères



La modification de données (1)

■ La syntaxe générale

UPDATE nom_table_cible

SET colonne = valeur

[, colonne2 = valeur2 ...]

[**WHERE** condition]

[**RETURNING** *];

[] *optionnel dans la requête*

La modification de données (2)

■ Mise à jour sans condition

Attention modification de ou des colonnes spécifiées sur toutes les lignes de la table

// mise à jour du tarif

```
UPDATE TARIF
```

```
SET PETIT_DEJEUNE = 10 ;
```

//majoration de 15% du tarif

```
UPDATE TARIF
```

```
SET PETIT_DEJEUNE = PETIT_DEJEUNE * 1.15;
```

La modification de données (3)

■ Mise à jour sans condition

Attention modification de ou des colonnes spécifiées sur toutes les lignes de la table

UPDATE CLIENT

SET nom= **UPPER**(nom),
 prenom = **UPPER**(prenom),
 enseigne = **UPPER**(enseigne);

Ou

UPDATE CLIENT

SET (nom, prenom, enseigne) =(**UPPER**(nom), **UPPER**(prenom), **UPPER**(enseigne)) ;

La modification de données (4)

■ Mise à jour conditionnelle

```
UPDATE TARIF  
SET  PETIT_DEJEUNE = PETIT_DEJEUNE * 1.15  
WHERE EXTRACT(YEAR FROM DATE_DEBUT) > 1999 ;
```

La modification de données (5)

■ Mise à jour et les sous requêtes

```
UPDATE PROSPECT  
SET  nom = 'bis_' || nom  
WHERE prp_id IN (SELECT cli_id FROM CLIENT)  
RETURNING nom , prenom , email;
```

prenom	nom	email
MARY	bis_SMITH	MARY.SMITH@sakilacustomer.org
PATRICIA	bis_JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org

Modifie le nom si le l'identifiant des prospects est dans celui de client
Retourne le nom et prénom des prospects modifiés

La modification de données (6)

■ Mise à jour avec sous requête corrélée

```
UPDATE PROSPECT AS P  
SET nom= ( SELECT nom  
           FROM CLIENT  
           WHERE id_cli=P.prp_id);
```



Le nom du prospect est égal à celui du client dont l'identifiant est égal à celui du prospect courant

La modification de données (7)

- Il est possible faire une "jointure" avec une autre table, la syntaxe est une peu différente de la jointure classique

```
UPDATE courir C
```

```
SET positionarrivee = 'A'
```

```
FROM pilote P
```

```
WHERE C.idpilote = P.idpilote
```

```
AND nompilote='Alonso';
```

A decorative vertical pattern on the left side of the slide, composed of various colored hexagons (blue, yellow, black, white) and dots.

FIN

Le Langage SQL

Compléments sur la création de tables

Le Langage de Définition de Données (DDL)

A decorative graphic on the left side of the slide, consisting of a vertical arrangement of blue and yellow hexagons of various sizes, some solid and some outlined.

Introduction

- **Le langage de définition de données permet de créer la structure des tables qui composent votre schéma de données.**
- **Un schéma de données est issu d'un modèle conceptuel de données ou de tout autre modèle ou langage (UML par exemple) permettant la modélisation des données d'une application.**

La création de tables

■ La syntaxe générale

```
CREATE TABLE nom_table
(
  nom_colonne { type } [ DEFAULT valeur_default ][ contrainte.. ],
  .... ,
  CONSTRAINT nom_contrainte { UNIQUE |
                                PRIMARY KEY ( liste_colonne )|
                                FOREIGN KEY liste_colonne
                                REFERENCES nom_table (liste_colonne)}
);
```

La création de tables

- A sa création, la table peut être remplie par une requête SELECT.
Par défaut, une table est vide à sa création.

Exemple :

-- *changement de nom des colonnes sélectionnées, même type*

```
CREATE TABLE Marque_Voiture  
(    fabricant,  
    Modele  
) AS SELECT MARQUE , MODELE FROM VOITURE;
```

-- *même nom des colonnes sélectionnées, même type*

```
CREATE TABLE Marque_Voiture_1  
AS SELECT MARQUE , MODELE FROM VOITURE;
```

-- *Création de la copie d'une table*

```
CREATE TABLE Save_Voiture  
AS SELECT * FROM VOITURE;
```

Les types particuliers de colonne

- **il existe plusieurs types pour définir les colonnes d'une table :**
 - **Le type énumération**
 - **Le type tableau**
 - **Création de domaine de valeur**
 - **Les types composites**

Le type des colonnes

■ Les types énumération

- **Déclaration de types énumérés**

```
CREATE TYPE type_couleur AS ENUM ('bleu', 'vert', 'rouge');
```

ATTENTION, les valeurs de la listes ne peuvent être que des varchar !!!!

- **Utilisation d'un type ENUM**

```
CREATE TABLE voiture  
( id serial primary key,  
  modele varchar(45),  
  couleur type_couleur  
);
```

```
INSERT INTO voiture VALUES (default, 'focus', 'rouge');  
INSERT INTO voiture VALUES (default, 'focus', 'bleu');  
INSERT INTO voiture VALUES (default, 'ibiza', 'vert');  
INSERT INTO voiture VALUES (default, 'focus', 'noir'); -- erreur
```

id	modele	couleur
1	focus	rouge
2	focus	bleu
3	ibiza	vert

Erreur SQL :

ERREUR: valeur en entrée invalide pour le enum type_couleur : « noir »

Le type des colonnes

■ Les types énumération

```
..... AS ENUM ('bleu', 'vert', 'rouge');
```

- Tri sur des types énumérés

```
SELECT * FROM voiture WHERE couleur > 'bleu';
```

id	modele	couleur
1	focus	rouge
3	ibiza	vert

```
SELECT * FROM voiture WHERE couleur > 'bleu' order by couleur;
```

id	modele	couleur
3	ibiza	vert
1	focus	rouge

```
SELECT * FROM voiture  
WHERE couleur = (SELECT MAX(couleur)FROM voiture);
```

id	modele	couleur
1	focus	rouge

Le type des colonnes

■ Les types tableaux

- Déclaration de type tableau

```
CREATE TABLE sal_emp
(
    nom text,
    paye_par_semaine integer[], -- tableau à une dimension
    planning text[][]           -- tableau à deux dimensions
);
```

- Insertion de valeurs dans un type tableau

```
INSERT INTO sal_emp VALUES ('Bill',
                              '{10000, 10000, 10000, 10000}',
                              '{{"rendez-vous", "repas"}, {"entraînement", "présentation"}}');

INSERT INTO sal_emp VALUES ('Carol',
                              '{20000, 25000, 25000, 25000}',
                              '{{"petit-déjeuner", "consultation"}, {"rendez-vous", "repas"}}');
```

Le type des colonnes

■ Les types tableaux

- Insertion de valeurs dans un type tableau

Avec le constructeur ARRAY

```
INSERT INTO sal_emp VALUES ('David',  
                             ARRAY[10450, 8300, 6700, 12340],  
                             ARRAY[['cinéma', 'revue projet'], ['production', 'diner']]);
```

- Affichage du contenu de la table

```
SELECT * FROM sal_emp;
```

nom	paye_par_semaine	planning
Bill	{10000,10000,10000,10000}	{{rendez-vous,repas},{entrainement,présentation}}
Carol	{20000,25000,25000,25000}	{{petit-déjeuner,consultation},{rendez-vous,repas}}
David	{10450,8300,6700,12340}	{{cinéma,"revue projet"},{production,diner}}

Le type des colonnes

■ Les types tableaux

- Accéder à un élément du contenu de la table
Attention les indices commencent à 1

```
SELECT paye_par_semaine, paye_par_semaine[1]
FROM sal_emp;
```

paye_par_semaine	paye_par_semaine
{10000,10000,10000,10000}	10000
{20000,25000,25000,25000}	20000
{10450,8300,6700,12340}	10450

```
SELECT planning,planning[1][2],planning[2][1]
FROM sal_emp
```

planning	planning	planning
{{rendez-vous,repas},{entrainement,présentation}}	repas	entrainement
{{petit-déjeuner,consultation},{rendez-vous,repas}}	consultation	rendez-vous
{{cinéma,"revue projet",{production,dîner}}	revue projet	production

Pour en savoir plus sur la manipulation des types tableaux :

<http://docs.postgresqlfr.org/9.1/arrays.html>

Le type des colonnes

■ Création d'un domaine de valeur

Un domaine est essentiellement un type de données avec des contraintes optionnelles (restrictions sur l'ensemble de valeurs autorisées)

Syntaxe :

```
CREATE DOMAIN nom [AS] type_donnee  
    [ DEFAULT expression ]  
    [ constraint [ ... ] ]
```

où **constraint** est :

```
[ CONSTRAINT nom_contrainte ]  
{ NOT NULL | NULL | CHECK (expression) }
```

Le type des colonnes

■ Création d'un domaine de valeur

Exemple :

Définition

```
CREATE DOMAIN DOM_tel AS varchar(20)
    constraint tel_val
        check (VALUE SIMILAR TO '[0-9][0-9]-[0-9][0-9]-[0-9][0-9]-[0-9][0-9]-[0-9][0-9]');
```

Utilisation

```
CREATE TABLE client
(id_cli int primary key,
 nom_cli varchar(25),
 prenom_cli varchar(25),
 tel_cli DOM_tel );
```

Le type des colonnes

■ Les types composites

Un type composite s'apparente à une structure de données utilisés dans différents langages de programmation.

Création d'un type composé : syntaxe générale

```
CREATE TYPE nom_type AS (  
    nom_colonne1 { type },  
    nom_colonne2 { type }, ..... );
```

*Impossible de définir des contraintes sur les colonnes lors de la déclaration du type,
il faudra définir un domaine de valeur pour définir une contrainte*

Exemple

```
CREATE TYPE complexe AS  
( r real,  
  i real);
```

Le type des colonnes

■ Les types composites

Un type composite s'apparente à une structure de données utilisés dans différents langages de programmation.

Utilisation d'un type composé

- Exemple de déclaration de colonne avec le type composé:

```
CREATE TABLE nombre  
    ( nombre_C complexe,  
      nombre_Ent integer );
```

- Exemple d'insertion de données dans une colonne de type composé:

```
INSERT INTO nombre VALUES (ROW(0.56, 1.99), 3);
```

- Exemple de modification d'une colonne de type composé

```
UPDATE nombre  
SET nombre_c = ROW(0.75, 0.85) ;
```

Le type des colonnes

- Les types composites

Utilisation d'un type composé

Exemple d'affichage des colonnes du type composé :

```
SELECT (nombre_C).r as "Réal", (nombre_C).i as "Imaginaire"  
FROM nombre;
```

Réal	Imaginaire
0.75	0.85

Le type des colonnes

- Les types composites

Exemple de suppression du type composé :

Drop type complexe ;

Le type des colonnes

- Les types composites pour une table : définir une table de ce type

Exemple :

```
CREATE TYPE type_employe AS (id int, nom text, salaire numeric);
```

```
CREATE TABLE employes OF type_employe (  
    PRIMARY KEY (id),  
    nom WITH OPTIONS DEFAULT 'Dirlo',  
    salaire WITH OPTIONS DEFAULT 1000  
);
```

Il est impossible de définir des contraintes sur les colonnes d'un type composite lors de sa définition => WITH OPTIONS ...

Si on supprime le type composé on doit supprimer la table qui est de ce type : *DROP type_employe CASCADE*

A vertical decorative element on the left side of the slide, composed of a cluster of small blue, yellow, and black hexagons.

MODIFICATION DE LA STRUCTURE D'UNE TABLE

A decorative element on the right side of the slide, consisting of a cluster of blue, yellow, and black hexagons, some of which are larger and more prominent than others.

Modification et Suppression

■ Ajout d'une colonne

Syntaxe :

```
ALTER TABLE relation  
    ADD COLUMN nom_colonne type [contrainte]
```

ou

```
ALTER TABLE relation  
    ADD nom_colonne type [contrainte]
```

Ajoutons l'attribut *fax* et l'attribut *ville*:

```
ALTER TABLE T_PERSONNE  
    ADD prs_fax DECIMAL(10,0),  
    ADD prs_ville varchar(10) ;
```

Les nouvelles colonnes sont placées à la fin de la table

Modification et Suppression

■ Supprimer une colonne

Attention, supprimer un attribut implique la suppression des valeurs qui se trouvent dans la colonne qui correspond à cet attribut.

Syntaxe :

```
ALTER TABLE relation DROP COLUMN attribut;
```

ou

```
ALTER TABLE relation DROP attribut;
```

Exemple :

```
ALTER TABLE T_PERSONNE DROP COLUMN prs_Ville;
```

Modification et Suppression

■ Ajout/suppression d'une contrainte de clé étrangère

Ajout :

```
ALTER TABLE T_TELEPHONE  
    ADD CONSTRAINT FK_TEL_CLI FOREIGN KEY (CLI_ID)  
    REFERENCES T_CLIENT(CLI_ID);
```

Suppression :

```
ALTER TABLE T_TELEPHONE  
    DROP CONSTRAINT FK_TEL_CLI;
```

Modification et Suppression

■ Ajout/suppression d'une clé primaire

Ajout :

```
ALTER TABLE Personnes ADD PRIMARY KEY (nom,prenom)
```

Supression :

```
ALTER TABLE Personnes DROP constraint Personnes_pkey ;
```

ATTENTION : Vous devez chercher dans le schéma de la base de données le nom de la contrainte qui défini la clé primaire.

Par défaut ce nom est composé du nom de la table suivi de 'pkey'

Exemple : si la clé est dans la tables *t_personne* alors la contrainte de clé primaire ce nomme *t_personne_pkey*

Modification et Suppression

- **Changer le nom et/ou le type d'une colonne**

Changer de nom de la colonne CLI_PRENOM de la table T_CLIENT

```
ALTER TABLE T_CLIENT  
        RENAME COLUMN CLI_PRENOM TO PRENOM;
```

Changer le type de la colonne PRENOM sans la renommer

```
ALTER TABLE T_CLIENT  
        ALTER COLUMN PRENOM TYPE VARCHAR(20);
```

Modification et Suppression

- Ajouter une valeur à un type énuméré

```
ALTER TYPE nom ADD VALUE [ IF NOT EXISTS ]  
nouvelle_valeur_enumérée [ { BEFORE | AFTER } valeur_enumérée ]
```

Exemple :

```
ALTER TYPE type_couleur ADD VALUE IF NOT EXISTS 'jaune';
```

Modification et Suppression

■ Modifier/Supprimer la valeur par défaut d'une colonne

Modifie la valeur par défaut de la colonne CLI_ENSEIGNE de la table T_CLIENT

```
ALTER TABLE T_CLIENT  
    ALTER CLI_ENSEIGNE SET DEFAULT 'PARTICULIER';
```

Supprime la valeur par défaut de la colonne CLI_ENSEIGNE

```
ALTER TABLE T_CLIENT  
    ALTER CLI_ENSEIGNE DROP DEFAULT ;
```

Modification et Suppression

■ Renommer une table

```
ALTER TABLE T_CLIENT  
    RENAME TO CLIENT ;
```

Pour en savoir plus sur la modification de la structure d'une table :

<http://docs.postgresqlfr.org/8.3/sql-altertable.html>

■ Supprimer une table

```
DROP {TABLE | VIEW } NOM_OBJET ;
```

Exemple :

```
DROP TABLE personne ;
```



FIN

1. CORRECTION D'ERREURS D'INSERTION

a. **INSERT INTO formateur VALUES(1, 'Jacques' , 'Mesrine');**

Erreur SQL :

ERROR: duplicate key value violates unique constraint "formateur_pkey"
DETAIL: Key (id_formateur)=(1) already exists.

La clé primaire existe déjà, il faut changer sa valeur :

INSERT INTO formateur VALUES(DEFAULT, 'Jacques' , 'Mesrine');

L'utilisation du **DEFAULT** génère une valeur auto incrémentée, puisque la colonne **id_formateur** est de type **SERIAL**

b. **INSERT INTO inscrire VALUES(10, 'X00005',1);**

Erreur SQL :

ERROR: insert or update on table "inscrire" violates foreign key constraint "inscrire_id_formation_fkey"
DETAIL: Key (id_formation)=(10) is not present in table "formation".

La formation d'identifiant N°10 n'existe pas, violation de la clé étrangère, il faut mettre une valeur dans colonne clé étrangère **id_formation** qui correspond à une valeur existante dans la colonne clé primaire (**id_formation**) de la table **formation** :

INSERT INTO inscrire VALUES(42, 'X00005',1);

c. **INSERT INTO inscrire VALUES(42, 1234556,1);**

Erreur SQL :

ERROR: insert or update on table "inscrire" violates foreign key constraint "inscrire_id_stagiaire_fkey"
DETAIL: Key (id_stagiaire)=(1234556) is not present in table "stagiaires".

Le stagiaire d'identifiant N°1234556 n'existe pas, violation de la clé étrangère, il faut mettre une valeur dans colonne clé étrangère **id_stagiaire** qui correspond à une valeur existante dans la colonne clé primaire (**id_stagiaire**) de la table **stagiaire**

INSERT INTO inscrire VALUES(42, 13167,1);

d. **INSERT INTO plannifier VALUES(42, 6 , '2019-01-23', NULL, 2, 1, 'Matin', 'E410');**

Erreur SQL :

ERROR: null value in column "groupe" violates not-null constraint
DETAIL: Failing row contains (42, 6, 2019-01-23, null, 2, 1, Matin, E410).

La colonne **groupe** ne peut être vide (NULL) il faut mettre une valeur dans cette colonne.

INSERT INTO planifier VALUES(42, 6 , '2019-01-23', 1, 2, 1, 'Matin', 'E410');

2. MODIFICATION DE LA DATE D'UNE FORMATION

--ajouter 9 jours à la date des formations

```
UPDATE planifier
SET Dateform = Dateform + INTERVAL '9 DAY'
WHERE id_formation=(SELECT id_formation
                     FROM formation
                     WHERE intitule_formation like '%Initiation a Linux%');
```

Ou avec les pseudos jointures

```
UPDATE planifier P
SET Dateform = Dateform + INTERVAL '9 DAY'
FROM formation F
WHERE F.id_formation=P.id_formation
AND intitule_formation like '%Initiation a Linux%';
```

La fonction INTERVAL permet d'ajouter ou retrancher un intervalle à une date, intervalle en jour (DAY), mois (MONTH), année (YEAR) et bien d'autre, voir la documentation pour connaître l'ensemble des intervalles possibles

3. SUPPRESSION DE LA FORMATION "BASES DE DONNEES (Access)"

Avant de supprimer la formation "Bases de données (Access)" il faut supprimer toutes les lignes des tables inscrire et planifier qui y font références

```
-- suppression de toutes les inscriptions à cette formation de la table inscrire
DELETE FROM inscrire
WHERE id_formation = (SELECT id_formation
                     FROM formation
                     WHERE intitule_formation like '%Bases de donnees (Access)%');
```

Ou avec les pseudos jointures

```
DELETE FROM inscrire I
USING formation F
WHERE F.id_formation=I.id_formation
AND intitule_formation like '%Bases de donnees (Access)%';
```

```
-- suppression de la formation du planning (table planifier)
DELETE FROM planifier
WHERE ID_FORMATION = (SELECT ID_FORMATION
                     FROM formation
                     WHERE intitule_formation like '%Bases de donnees (Access)%') ;
```

Ou avec les pseudos jointures

```
DELETE FROM planifier P
USING formation F
WHERE F.id_formation=P.id_formation
AND intitule_formation like '%Bases de donnees (Access)%';
```

```
-- suppression de la formation de la liste des formations (table formation)
DELETE FROM formation
WHERE intitule_formation like '%Bases de donnees (Access)%' ;
```

s

4. CONVERSION EN MINUSCULE

```
UPDATE formateur
SET nom_formateur = lower(nom_formateur),
    prenom_formateur = lower(prenom_formateur);
```

5. CONVERSION EN MAJUSCULE

```
UPDATE formateur
SET nom_formateur = UPPER(nom_formateur),
    prenom_formateur = initcap(prenom_formateur);

-- vérification de la modification
SELECT * FROM formateur ;
```

6. MODIFICATION DE SALLE

```
-- Affiche les formations prévues dans les 30 jours qui suivent le 8 avril 2006
SELECT *,(dateform -date '2006-04-08') as "ecart"
from planifier
where (dateform -date '2006-04-08') between 0 and 30;

-- modifie les dates de formation
Update planifier
Set numsalle='G333'
where (dateform -date '2006-04-08') between 0 and 30;
ou
Update planifier
Set numsalle='G333'
where dateform between date '2006-04-08' and date '2006-04-08'+ 30;

-- vérifie que les modifications sont réalisées
SELECT *,(dateform - date '2006-04-08') as "ecart"
from planifier
where (dateform - date '2006-04-08') between 0 and 30;
```

7. MODIFICATION DE LA DATE DE LA FORMATION

```
-- Affiche les formations qui ont lieu le 139ème jour de l'année
SELECT *, EXTRACT(DOY FROM dateform ) as "Numéro du jour de l'année 2006"
from planifier
where EXTRACT(DOY FROM dateform ) =139
AND extract (year from dateform)=2006 ;

-- repousse de 3 jours les formations qui ont lieu le 139ème jour de l'année
update planifier
set dateform=dateform + INTERVAL '3 day'
where EXTRACT(DOY FROM dateform ) =139
AND extract (year from dateform)=2006 ;

-- vérifie que les modifications sont réalisées
SELECT *, EXTRACT(DOY FROM dateform )as Num_jour_annee
from planifier
where EXTRACT(DOY FROM dateform ) =142
AND extract (year from dateform)=2006 ;
```

8. DEPART D'UN FORMATEUR, QUI SERA REMPLACÉ PAR UN NOUVEAU

- A. Affichez le planning des formations, on verra apparaître (en plus des informations de la table "planifier") le nom et prénom des formateurs ainsi que l'intitulé des formations qu'ils assurent.

```
SELECT intitule_formation, nom_formateur, prenom_formateur,  
to_char(dateform, 'DD/MM/YYYY') as "DATE", planifier.*  
FROM formation NATURAL JOIN planifier NATURAL JOIN formateur;
```

- B. Le formateur Cancel Christophe est malade, il ne peut donc plus assurer les formations qu'il a en charge. Un nouveau formateur est recruté (Durant Pierre) pour remplacer Cancel Christophe le temps de sa convalescence.

Vous devez :

- a. Insérer Durant Pierre dans la base données; -- default pour l'auto incrémentation de l'id

```
INSERT INTO formateur VALUES( default , 'Durant', 'Pierre');
```

- b. Modifier le planning pour que les formations assurées par Cancel Christophe soit assurée par Durant Pierre ;

-- toutes les formations assurées par Cancel sont maintenant assurées par Durant, il faut donc changer la valeur de la
-- clé étrangère de la table inscrire

```
UPDATE planifier  
SET id_formateur = (SELECT id_formateur FROM formateur  
WHERE nom_formateur LIKE 'Durant'  
AND prenom_formateur LIKE 'Pierre')  
WHERE id_formateur =(SELECT id_formateur FROM formateur  
WHERE nom_formateur LIKE 'CANCEL'  
AND prenom_formateur LIKE 'Christophe') ;
```

- C. Afficher le planning des formations afin vérifier que le changement à bien été effectué

Reprendre la requête du A

9. AJOUT D'UNE FORMATION ET INSCRIPTION

- A. Ajouter une formation intitulée "Javascript" d'une durée de 12 h niveau Confirme
 -- insertion de la nouvelle formation, l'identifiant et auto incrémenté default permet d'ajouter 1 à la dernière
 --valeur de la colonne id_formation

```
INSERT INTO formation VALUES ( default, 'Javascript', 12, 'Confirme' )
```

- B. Tous les inscrits à la formation "Excel" niveau "Débutant" désirent participer à la formation précédemment créée. Le groupe de chaque professeur est identique. Réaliser les requêtes qui permettent de satisfaire cette demande.

Conseil : vous pouvez créer une table "tampon" qui sera détruite à la fin
 -- recherchons les inscrits à la formation excel
 niveau debutant

```
SELECT id_formation , id_stagiaire, groupe
FROM inscrire natural join formation
WHERE intitule_formation LIKE '%Formation Excel%'
AND niveau LIKE '%Debutant%' ;
```

L' id_formation 26 correspond à l'id de la formation Excel, avant d'insérer ces lignes dans la table inscrire, il suffit donc de changer cette id par celui de javascript, nous placerons une sous requête à la place de la colonne id_formation après le SELECT

```
SELECT (SELECT max(id_formation) from formation) ,
id_stagiaire, groupe
FROM inscrire natural join formation
WHERE intitule_formation LIKE '%Formation Excel%'
AND niveau LIKE '%Debutant%' ;
```

La sous requête SELECT max(id_formation) from formation retourne la valeur numérique la plus grande (la dernière entrée)

Il suffit maintenant d'insérer ces lignes dans la table inscrire

```
INSERT INTO inscrire
SELECT (SELECT max(id_formation) from formation) ,
id_stagiaire, groupe
FROM inscrire natural join formation
WHERE intitule_formation LIKE '%Formation Excel%'
AND niveau LIKE '%Debutant%' ;
```

```
-- verification
SELECT id_formation , id_stagiaire, groupe
FROM inscrire natural join formation
WHERE intitule_formation LIKE 'Javascript' ;
```

id_formation	id_stagiaire	groupe
26	10600	1
26	12456	1
26	13073	1
26	13150	1
26	13516	1
26	13677	1
26	14186	1
26	14487	1
26	14899	1
26	14918	1
26	15118	1
26	19362	1
26	21435	2
26	23248	2
26	27053	2
26	29239	2
26	29428	2
26	3803	2
26	38424	2
26	41078	2
26	41181	2
26	41206	2
26	4453	2
26	44961	1
26	6564	2
26	6725	2

26 ligne(s)

10. SUPPRESSION DES FORMATIONS NON DISPENSEES AU SEIN DU LYCEE

- A. Afficher toutes les formations (toutes les colonnes) qui ne sont pas assurées au sein du lycée (nombre d'heures égal à 0)

```
SELECT *
FROM formation
WHERE Nbheures =0;
```

- B. Afficher toutes les colonnes de la table *inscrire*, mais seulement les lignes qui correspondent aux inscrits aux formations qui ne sont pas assurées au sein du lycée (nombre d'heures égal à 0).

```
SELECT I.*
FROM inscrire I inner join formation F ON F.id_formation=I.id_formation
WHERE nbheures=0;
```

Ou

```
SELECT inscrire.*
FROM inscrire natural join formation
WHERE nbheures=0;
```

- C. Vous devez supprimer de la base de données les formations qui ne sont pas assurées au sein du lycée et les inscrits à ces formations, mais avant de les supprimer nous aimerions avoir une sauvegarde de ces données :

- a. En utilisant la syntaxe donnée dans les remarques :
- Créez une table nommée *Formation_Ext* en y insérant les formations qui ne sont pas assurées au sein du lycée (nombre d'heures égal à 0)

```
-- création de la table Formation_Ext et insertion des formations extérieures
```

```
CREATE TABLE Formation_Ext
AS SELECT * FROM formation WHERE nbheures =0;
```

- Créez une table *Inscrit_Form_Ext* en y insérant les inscrits aux formations qui ne sont pas assurées au sein du lycée (nombre d'heures égal à 0)

```
-- Création de la sauvegarde des inscriptions aux formations extérieures
```

```
CREATE TABLE Inscrit_Form_Ext
AS SELECT inscrire.*
FROM inscrire natural join formation
WHERE nbheures=0;
```

- b. Une fois les sauvegardes réalisées supprimez les données correspondantes des tables d'origines ;

```
-- Suppression des tuples qui correspondent aux inscrits aux formations extérieures
```

```
DELETE FROM inscrire
WHERE id_formation in (SELECT id_formation
FROM formation
WHERE nbheures =0);
```

```
-- Suppression de 44 lignes
```

Commentaire : Suppression des tuples de la table "suivreformation" dont l'identifiant est dans la liste (IN) des identifiants des formations qui ne sont pas assurées (Nbheures =0)

```
-- Suppression des tuples qui correspondent aux formations non assurées par le lycée
```

```
delete from formation
where nbheures =0;
```

```
-- Suppression de 4 lignes
```

Correction TP2

1 PREMIERE ETAPE : CREATION ET MODIFICATION D'ATTRIBUTS

1.1. Ajout de colonne

On souhaite intégrer dans la relation *planning* un deuxième tarif de voyage pour les enfants. Effectuer l'ajout de cet attribut *tarifenf* dans la relation en utilisant le même type de données que pour la colonne *tarif*. De manière similaire, le nombre d'enfants doit être ajouté dans la relation *reservation*. Donnez à cet attribut *nbenf* le même type que celui de *nbpers*.

```
ALTER TABLE planning ADD tarifenf numeric(6,2) ;
ALTER TABLE reservation ADD nbenf numeric(2,0) ;
```

1.2. Modification de type d'une colonne

On souhaite étendre la définition du type de l'attribut *libelle* de la table *optionv*, en augmentant la taille du nombre de caractères admissibles de 10 caractères. Consultez la définition de cet attribut puis effectuez la modification correspondante et vérifiez-la.

```
ALTER TABLE optionv ALTER libelle TYPE VARCHAR(30) ;
```

1.3. Contrainte de vérification de domaine

1. On souhaite intégrer dans la base une contrainte de domaine pour l'attribut *categorie* de la table *client* qui prendra ses valeurs dans l'ensemble suivant : {PRIVILEGIE, BON, MAUVAIS}. Effectuez cet ajout de contrainte.

```
CREATE DOMAIN DOM_cat_cli AS varchar(15)
CONSTRAINT DOM_CATEGORIE
CHECK (VALUE IN ('PRIVILEGIE', 'BON', 'MAUVAIS')) ;

-- Ensuite il faut modifier le type de la colonne CATEGORIE de la table CLIENT
ALTER TABLE CLIENT
ALTER CATEGORIE TYPE DOM_cat_cli;
```

2. Modifier la valeur par défaut de la colonne l'attribut *categorie* de la table *client* pour qu'elle soit MAUVAIS

```
ALTER table CLIENT
ALTER categorie SET DEFAULT 'MAUVAIS';

Update CLIENT
Set categorie ='MAUVAIS'
Where categorie is NULL;
```

3. Spécifiez une contrainte de domaine pour l'attribut *nbetoiles* dans la table *voyage* afin que ses valeurs appartiennent à l'intervalle [1 .. 5].

```
ALTER TABLE VOYAGE
ADD CONSTRAINT DOM_NBETOILES CHECK (NBETOILES BETWEEN 1 AND 5) ;
```

1.4. Ajout des tables *capacite* et *hotel*

```
CREATE TYPE type_chambre AS ENUM ('SINGLE', 'DOUBLE', 'DOUBLE LUXE',
'SUITE', 'SUITE JUNIOR', 'SUITE PRESTIGE');

CREATE TABLE capacite
(
    id_hotel integer,
    typechambre type_chambre,
    nbrch NUMERIC(3,0),
    CONSTRAINT PK_CAPACITE PRIMARY KEY(id_hotel,typechambre)) ;

CREATE TABLE HOTEL
(
    id_hotel SERIAL primary key,
    nomHotel VARCHAR(20) default NULL,
    NbEtoiles NUMERIC(1,0) default NULL,
    VilleHotel VARCHAR(20) default NULL,
    CONSTRAINT DOM_NBETOILES CHECK (NBETOILES BETWEEN 1 AND 5)) ;
```

```

-- création de la clé étrangère dans la table capacite
ALTER TABLE capacite
ADD CONSTRAINT FK_hotel FOREIGN KEY(id_hotel) REFERENCES HOTEL(id_hotel)
    ON delete CASCADE;

-- ajout de la clé étrangère dans la table voyage
ALTER TABLE voyage
ADD id_hotel integer,
ADD CONSTRAINT FK_Voy_hotel FOREIGN KEY(id_hotel) REFERENCES HOTEL(id_hotel) ;

-- transfère des données de la table VOYAGE vers dans la table HOTEL
-- On ne s'occupe pas de la colonne id_hotel qui est auto incrémenté (SERIAL)
INSERT INTO hotel(nomHotel, NbEtoiles,VilleHotel)
SELECT DISTINCT hotel, nbetoiles,villearr
FROM voyage

-- Valorisation de la colonne id_hotel de la table voyage, on utilise ici une
-- sous requête corrélée dans le update
UPDATE VOYAGE V
SET id_hotel=(SELECT id_hotel
              FROM hotel
              WHERE nomhotel=V.hotel AND villehotel=V.villearr)

-- vérification des valeurs avant de supprimer les colonnes de la table voyage
SELECT hotel=nomhotel as "hotel", villearr=villehotel as "ville"
FROM voyage V JOIN hotel H ON H.id_hotel=V.id_hotel

-- La comparaison dans le select renvoie TRUE si le nom des hôtels est correcte, FALSE
-- le cas contraire, idem pour la ville, cela permet une vérification visuelle rapide,
-- on peut faire plus rapide :

SELECT *
FROM voyage V JOIN hotel H ON H.id_hotel=V.id_hotel
WHERE NOT (hotel=nomhotel AND villearr=villehotel)

Ici, si la requête ne retourne aucune ligne, c'est que tout est bon !

-- on peut maintenant supprimer les colonnes de la table voyage
ALTER TABLE voyage
DROP villearr,
DROP hotel,
DROP nbetoiles ;

-- insertion des données dans la table capacite
-- On peut utiliser une table temporaire ou directement faire des insert
-- avec sous requête, je vais utiliser une table temporaire

CREATE TABLE temp_capacite
(
    hotel VARCHAR(20),
    typechambre type_chambre,
    nbrch NUMERIC(3,0)) ;

-- pour être compatible avec la colonne typechambre de la table capacite

```

```
-- insertion des données des hotels
INSERT INTO temp_capacite values
('ANTIQUE','SINGLE', 10),('ANTIQUE','DOUBLE',75),
('ANTIQUE','SUITE', 5),('ATLAS','DOUBLE',83),
('ATLAS','SUITE',27),('OLD BRIDGE','SINGLE',25),
('OLD BRIDGE','DOUBLE',75),('SAFARI JAMBO','SINGLE',32),
('SAFARI JAMBO','DOUBLE',100),('TRANSATLANTIQUE','DOUBLE',200),
('BAMBURI','DOUBLE',150),('BELLERIVE','DOUBLE',56),
('BELLERIVE','SUITE',12),('DAR AL BAHAR','SINGLE',10),
('DAR AL BAHAR','DOUBLE',12),('EL ALMARANTE','SINGLE',34),
('EL ALMARANTE','SUITE',45),('EL ANDALOUS','SINGLE',67),
('EL ANDALOUS','DOUBLE',54),('ELIAS BEACH','SINGLE',87),
('ELIAS BEACH','DOUBLE',33),('ESPADON','SINGLE',34),
('ESPADON','DOUBLE',45),('HASNA','SINGLE',32),
('HASNA','DOUBLE',24),('MONDIAL','SINGLE',12),
('MONDIAL','DOUBLE',13),('KENZI CLUB','SINGLE',45),
('KENZI CLUB','DOUBLE',67) ;
```

On peut aussi utiliser l'import du fichier csv..

```
-- il suffit ensuite de travailler sur cette table pour l'insertion dans la
-- table capacite, on utilise encore une sous requête corrélée
```

```
INSERT INTO capacite
SELECT (SELECT id_hotel FROM hotel WHERE nomhotel=T.hotel), typechambre , nbrch
FROM temp_capacite T
```

```
-- ATTENTION si vous avez créé un type énuméré pour le type de chambre vous devez
-- caster la colonne typechambre de la table temporaire
-- Ici je ne le fais pas car quand je créé la table temporaire, j'applique le type
-- type_chambre à la colonne typechambre
```

```
-- on supprime la table temporaire
DROP TABLE temp_capacite
```

2 DEUXIEME ETAPE : MISES A JOUR DES DONNEES

1. Mettez à jour l'attribut *nbenf* en lui affectant la valeur 2 pour toutes les réservations du client 2103. Modifiez cet attribut en lui donnant la valeur 1 pour les réservations du client *Thomas Jarolim*.

```
UPDATE reservation
SET nbenf = 2
WHERE numcl = 2103 ;
```

```
UPDATE reservation
SET nbenf = 1
WHERE numcl IN (SELECT numcl FROM client WHERE nom = 'JAROLIM'
AND prenom = 'THOMAS') ;
```

2. Pour tous les tuples de la relation *planning*, effectuez la modification de *tarifenf* en lui affectant la valeur de *tarif* réduite de moitié.

```
UPDATE planning SET tarifenf = TARIF / 2 ;
```

3. Insérez deux nouveaux tuples dans la relation *client* en vérifiant la mise en œuvre de la contrainte de domaine sur *categorie*. Créez une réservation pour un de ces clients et un voyage existant.

```
INSERT INTO client (numcl, nom, prenom, ville)
VALUES ((select max(numcl)+ 1 from client),
'ALLEMAND', 'GREGORY', 'AIX-EN-PROVENCE') ;
INSERT INTO client
VALUES ((select max(numcl)+ 1 from client),
'BENSALAH', 'MALEK', NULL, NULL, 'MARSEILLE', NULL) ;
```

```
--L'insertion suivante échoue à cause de la CI de domaine
INSERT INTO client
VALUES((select max(numcl)+ 1 from client), 'BEUF', 'FRANCOIS',NULL,
      NULL, 'MARSEILLE', 'PREMIER') ;
```

Erreur SQL :

ERREUR: la nouvelle ligne viole la contrainte de vérification « client » de la relation « dom_categorie »
DETAIL: La ligne en échec contient (2307, BEUF, FRANCOIS, null, null, MARSEILLE, PREMIER)

En effet PREMIER n'est pas une valeur possible définit dans la contrainte de domaine DOM_cat_cli défini au 1.3.

4. Mettez à jour la relation voyage en tentant de donner une valeur invalide à l'attribut nbetoiles.

```
UPDATE VOYAGE SET NBETOILES = 9 ;
```

Erreur SQL :

ERREUR: la nouvelle ligne viole la contrainte de vérification « voyage » de la relation « dom_nbetoiles »
DETAIL: La ligne en échec contient (867.00, LISBONNE, PORTUGAL, MARSEILLE, MONDIAL, 9, 2)

1 MODIFICATION DU DIAGRAMME DE CLASSES

```
-- table: proprietaire
```

```
CREATE TABLE proprietaire (  
    numprop      serial      primary key,  
    nomprop      varchar(35) null,  
    adrprop      varchar(50) null,  
    cpprop       decimal(5)  null,  
    villeprop    varchar(50) null,  
    telprop      varchar(20) null );
```

```
-- table : activites
```

```
CREATE TABLE activites (  
    numact      serial      not null,  
    nomact      varchar(45) null,  
    CONSTRAINT pk_activites primary key (numact) );
```

```
-- table : organisme
```

```
CREATE TABLE organisme (  
    id_organisme serial      not null,  
    nom_organisme varchar(45) not null,  
    adr_organisme varchar(175) not null,  
    ville_organisme varchar(75) not null,  
    cp_organisme decimal(5,0) not null,  
    CONSTRAINT pk_organisme primary key (id_organisme) );
```

```
-- creation d'un domaine de valeur pour la colonne interet du type composé
```

```
--
```

```
CREATE DOMAIN dom_interet AS int  
    DEFAULT 1  
    CONSTRAINT ck_interet check(value between 1 and 5);
```

```
-- creation d'un type composé pour la colonne info de proposer
```

```
--
```

```
CREATE type information AS  
(  
    distance real,  
    interet dom_interet  
);
```

```
-- table : proposer
```

```
CREATE TABLE proposer (  
    numgite      int          NOT null,  
    numact       int          NOT null,  
    info         information   NOT null,  
    CONSTRAINT pk_proposer primary key (numgite, numact));
```

```
-- table : proposer_par
```

```
CREATE TABLE proposer_par (  
    numgite      int          not null,  
    numact       int          not null,  
    id_organisme int          not null,  
    CONSTRAINT pk_proposer_par primary key (numgite, numact ,id_organisme)  
);
```

```
-- clé étrangère de la table proposer
```

```
ALTER TABLE proposer  
    ADD CONSTRAINT fk_proposer_activite FOREIGN KEY (numact)  
        REFERENCES activites (numact) ON UPDATE CASCADE,  
    ADD CONSTRAINT fk_proposer_gite FOREIGN KEY (numgite)  
        REFERENCES gites (numgite) ON UPDATE CASCADE;
```

```
-- clé étrangère de la table proposer_par
-- ATTENTION à la clé étrangère composée de deux colonnes qui fait
-- référence à la table proposer
ALTER TABLE proposer_par
  ADD CONSTRAINT fk_pro_org FOREIGN KEY (id_orga)
    REFERENCES organisme (id_orga) ON UPDATE CASCADE,
  ADD CONSTRAINT fk_pr FOREIGN KEY (numgite, numact)
    REFERENCES proposer (numgite, numact) ON UPDATE CASCADE;

-- ajout de l'attribut clé étrangère et de la clé étrangère
-- dans la table gites
ALTER TABLE gites
  add numprop int not null,
  add FOREIGN KEY (numprop)
    REFERENCES proprietaire(numprop) ON UPDATE CASCADE;

-- ajout des attributs dans la table gites
ALTER TABLE gites
  add superficiegit int null,
  add nbepis int default 1,
  add nbpers int default 2;

-- ajout de l'attribut dans la table facturer
ALTER TABLE facturer
  add tarifttc decimal(6,2) not null;
```

2 SUPPRESSION D'ATTRIBUTS

```
ALTER table CLIENTS drop COLUMN NB_ENFANTS;
```

3 MODIFICATION DU TYPE D'UN ATTRIBUT

```
ALTER table GITES
alter column TELGITE TYPE varchar(15),
add constraint ck_tel check(TELGITE SIMILAR TO '[0-9][0-9]-[0-9][0-9]-[0-9][0-9]-[0-9][0-9]-[0-9][0-9]-[0-9][0-9]');
```

4 MODIFICATION DU NOM ET DU TYPE D'UN ATTRIBUT

```
-- renommer la table
ALTER TABLE GITES RENAME COLUMN DESCR TO description;

-- modifier le type
ALTER TABLE GITES ALTER COLUMN description TYPE VARCHAR(255);
```

5 AJOUT D'ENREGISTREMENTS

A. Ajout d'un nouveau gîte :

ATTENTION : à l'ordre dans lequel vous allez enregistrer les informations dans la base de données, il est important de commencer par les tables qui n'ont pas de clé étrangère :

1. PROPRIETAIRE : coordonnées du propriétaire
2. GITES : adresse du gîte, superficie, nombre de personnes et le nombre d'épis
3. CHAMBRES : caractéristiques de chaque chambre + le lien vers MAISON_INDIV
4. SAISON : ajout des saisons (elles seront ajoutées une seule fois)
5. FACTURER : le tarif pour chaque saison

```
-- AJOUT DU PROPRIETAIRE
INSERT INTO proprietaire
values(default , 'dusic pierre', '44, rue pierre curie',
```

```
94140, 'Alfortville', '01-45-02-10-20');
```

```
-- AJOUT DU GITE DANS LA TABLE GITES
```

```
INSERT INTO gites
```

```
values(default, 'les castors', '5 rue qui monte', 74100, 'st gervais',  
        '04-42-05-70-52',  
        'charmant gite de plein pied au cœur de la montagne',  
        (select max(numprop)from propriétaire) ,120, 3, 10);
```

```
-- AJOUT DES CHAMBRES ET DE LEURS CARACTERISTIQUES DANS LA TABLE CHAMBRES
```

```
-- la sous requête permet de récupérer l'identifiant du gite précédemment créé,  
-- pour associer la chambre à un gite
```

```
-- Chambre 1 : 1 lit simple, 1 lit double, superficie de 15m²;
```

```
INSERT INTO Chambres VALUES(default, (select max(numgite)from gites),1,1,15) ;
```

```
-- Chambre 2 : 1 lit double, superficie de 20m²;
```

```
INSERT INTO Chambres VALUES (default, (select max(numgite)from gites) ,0,1,20);
```

```
-- AJOUT DES SAISONS DANS LA TABLE SAISON
```

```
INSERT INTO saison VALUES ('BASSE') ;
```

```
INSERT INTO saison VALUES ('MOYENNE') ;
```

```
INSERT INTO saison VALUES ('HAUTE');
```

```
-- AJOUT DES TARIFS DANS LA TABLE FACTURER
```

```
-- la sous requête permet de récupérer l'identifiant du gite précédemment créé,  
-- pour associer les prix par saison à un gite
```

```
INSERT INTO facturer VALUES ('BASSE', (select max(numgite)from gites), 800);
```

```
INSERT INTO facturer VALUES ('MOYENNE', (select max(numgite)from gites), 1200);
```

```
INSERT INTO facturer VALUES ('HAUTE', (select max(numgite)from gites), 1800);
```

B. Ajout d'une réservation :

Le client Carmignac Laurent habitant au 4, allée des Camélias 13080 Luynes, téléphone 04-42-50-65-52, loue le nouveau gîte du 5/04/2005 pour une durée de 15 jours, cette location concerne 7 personnes

```
-- AJOUT DU CLIENT DANS LA TABLE CLIENTS
```

```
INSERT INTO clients VALUES(default, 'Carmignac Laurent',  
                              '4, allée des Camélias', 13080,  
                              'Luynes', '04-42-50-65-52');
```

```
-- AJOUT DES CARACTERISTIQUES DE LA RESERVATION DANS LA TABLE LOUER
```

```
INSERT INTO louer VALUES((select max(numgite)from gites),  
                          (select max(numcli)from clients),15,7,'2005-04-05');
```

Ecrire les requêtes SQL qui permettent de vérifier que les insertions se sont bien effectuées.

```
-- vérifie l'insertion du gite
SELECT *
FROM gites G JOIN chambres C ON C.numgite=G.numgite

-- vérifie l'insertion des tarifs
SELECT *
FROM gites G JOIN facturer F ON G.numgite=F.numgite

-- vérifie l'insertion de la location
SELECT *
FROM gites G JOIN louer L ON G.numgite=L.numgite
      JOIN clients C ON L.numcli=C.numcli
```

C. Ajout d'une nouvelle réservation :

Le même client que précédemment loue le même gîte du 2/07/2005 pour une durée de 3 semaines, cette location concerne 10 personnes

```
-- AJOUT DES CARACTERISTIQUES DE LA NOUVELLE RESERVATION DANS LA TABLE LOUER
INSERT INTO louer VALUES((select max(numgite)from gites),
                          (select max(numcli)from clients),21,10,'2005-07-02');
```

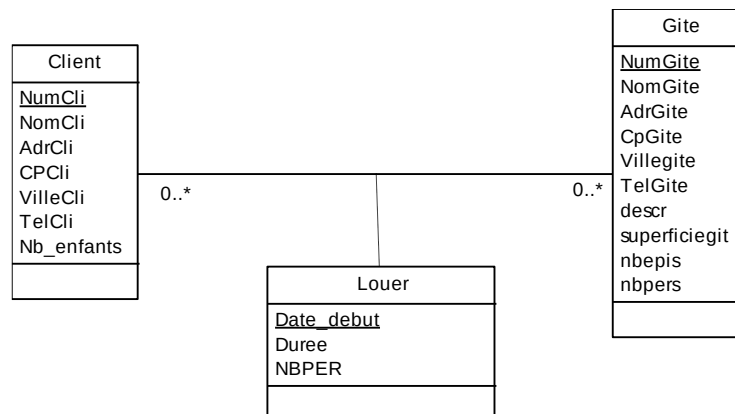
Il y a une erreur ?????

Erreur SQL :

ERREUR: la valeur d'une clé dupliquée rompt la contrainte unique « pk_louer »
DETAIL: La clé « (numgite, numcli)=(1, 1) » existe déjà.

Effectivement il est impossible pour un même client de louer le même gîte et ce quel que soit les dates, car la clé primaire de la table LOUER n'est composée que de l'identifiant de la table GITES et de celui de la table CLIENTS.

Il faut donc que la date de début de location (louer) soit clé primaire de la table LOUER, pour cela on modifie le diagramme de classes :



Dans la base de données, il suffit de supprimer la contrainte de clé primaire dans la table LOUER et d'en ajouter une nouvelle qui sera composée des id des tables GITES, CLIENTS et LOUER,

```
-- suppression de la cle primaire de la table louer
alter table louer drop constraint pk_louer;

-- ajout de la nouvelle la cle primaire de la table louer
alter table louer
add primary key (numgite, numcli, date_debut);
Vous pouvez maintenant ajouter l'enregistrement de la location
INSERT INTO louer VALUES((select max(numgite)from gites),
                          (select max(numcli)from clients),21,10,'2005-07-02');
```

6 AJOUT D'UNE ACTIVITE

Il faut associer l'activité *alpinisme* au gite précédemment saisi, cette activité se situe à 23 km du gite, l'intérêt est de 4 sur une échelle de 5.

Il y a deux organismes qui proposent cette activité :

Bureau des guides de Chamonix 190 Place de l'Église, 74400 Chamonix-Mont-Blanc

Les Guides Indépendants de Chamonix, 45 rue blanche, 74400 Chamonix-Mont-Blanc

```
-- ajout de l'activité
INSERT INTO activites VALUES(default, 'alpinisme');

-- ajout du lien entre l'activité et le gite(table proposer)
-- ATTENTION à l'insertion dans un type composé
INSERT INTO proposer VALUES((select max(numgite)from gites),
                             (select max(numact)from activites),
                             ROW(23,4));

-- ajout des organismes
INSERT INTO organisme VALUES(default, 'Bureau des guides de Chamonix',
                              '190 Place de l'Église','Chamonix-Mont-Blanc', 74400);
INSERT INTO organisme VALUES(default, 'Les Guides Indépendants de Chamonix',
                              '45 rue blanche','Chamonix-Mont-Blanc',74400);

-- ajout des liens entre les organisme et la table proposer
INSERT INTO proposer_par VALUES((select max(numgite)from gites),
                                 (select max(numact)from activites),
                                 (select max(id_orga)from organisme));

INSERT INTO proposer_par VALUES((select max(numgite)from gites),
                                 (select max(numact)from activites),
                                 (select min(id_orga)from organisme));

-- Verification des insertions

SELECT nomgite, nomact,nom_orga, (info).distance,(info).interet
FROM activites A JOIN proposer P ON P.numact=A.numact
                JOIN gites G ON G.numgite=P.numgite
                JOIN proposer_par Pp ON Pp.numgite=P.numgite
                                   AND Pp.numact=P.numact
                JOIN organisme O ON o.id_orga=Pp.id_orga
```

TRAVAIL A FAIRE

1. Ecrire et exécuter les requêtes de manipulation de données suivantes

1. Les prix (table couter : chb_prix) de toutes les chambres qui ont été réservées en 2001 est majoré de 33%.

```
UPDATE couter
SET chb_prix= chb_prix*1.33
WHERE chb_id IN (SELECT chb_id FROM reserver WHERE extract(year
FROM date_jour)=2001) ;
```

2. Mettre le nom des clients en majuscule et la première lettre du prénom en majuscule.

```
UPDATE client
SET nom=upper(nom),
    Prenom=initcap(prenom) ;
```

3. Le client dénommé PHILIPPE André (nom, prénom) nous a malheureusement quitté, il doit être supprimé de la base de données. Ecrire le(s) requête(s) qui permettent de réaliser cette suppression ;

```
DELETE FROM email
WHERE cli_id IN (SELECT cli_id
                 FROM client
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM adresse
WHERE cli_id IN (SELECT cli_id
                 FROM client
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM telephone
WHERE cli_id IN (SELECT cli_id
                 FROM client
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM ligne_facture
WHERE fac_id IN (SELECT fac_id
                 FROM facture join client using(cli_id)
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM facture
WHERE cli_id IN (SELECT cli_id
                 FROM client
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM reserver
WHERE cli_id IN (SELECT cli_id
                 FROM client
                 WHERE nom='PHILIPPE' AND prenom='André') ;
```

```
DELETE FROM client
WHERE nom='PHILIPPE' AND prenom='André' ;
```

4. Un nouveau client vient de réserver la chambre numéro 6 pour 2 personnes, il faut l'ajouter la base de donnée :

```
-- ajout du client
INSERT INTO client
VALUES (default,'m.','JENSUIS','Pierre',NULL);

-- ajout de son email
INSERT INTO email
VALUES(DEFAULT,
        (SELECT cli_id
         FROM client
         WHERE nom='JENSUIS' AND prenom='Pierre'),
        'jensuis.pierre@icemail.fr',
        'bureau') ;

-- ajout de son telephone
INSERT INTO telephone
VALUES(DEFAULT,
        (SELECT cli_id
         FROM client
         WHERE nom='JENSUIS' AND prenom='Pierre'),
        'tel',
        '04-45-67-89-32',
        'domicile') ;

-- ajout de son adresse
INSERT INTO adresse
VALUES(DEFAULT,
        (SELECT cli_id
         FROM client
         WHERE nom='JENSUIS' AND prenom='Pierre'),
        '34 rue noir',
        DEFAULT,
        DEFAULT,
        DEFAULT,
        34678,
        'busez') ;

-- ajout de sa réservation
INSERT INTO reserver
VALUES( (select chb_id
        from chambre
        where numero=6),
        current_date,
        (select cli_id
         From client
         Where nom='JENSUIS' and prenom='Pierre'),
        2,
        default,
        default);
```

2. Modification de la structure de la base de données

1. On doit supprimer de la table `ligne_facture` la colonne `remise_pourcent`, mais avant il faut mettre à jour la colonne `remise_montant` qui correspond au montant multiplié par la remise en pourcentage. Attention cette mise à jour se fera que quand la `remise_montant` est NULL

Ecrire le(s) requête(s) qui permettent de réaliser ces opérations ;

-- mise à jour de la colonne `remise_montant`

```
UPDATE ligne_facture
SET remise_montant = montant *(remise_pourcent/100)
WHERE remise_montant IS NULL ;
```

-- suppression de la colonne `remise_pourcent`

```
Alter table ligne_facture
Drop column remise_pourcent ;
```

2. La colonne `couchage` de la table `chambre` doit changer de nom et de type, elle doit s'appeler `nb_couchage` et son type doit être un numérique de 2 chiffres sans décimal. Écrire la(les) requête(s) qui permet(tent) de réaliser cette opération ;

-- on renomme la colonne

```
ALTER TABLE chambre
        RENAME COLUMN couchage TO nb_couchage;
```

-- on change son type

```
ALTER TABLE chambre
        ALTER COLUMN nb_couchage TYPE numeric(2,0);
```

3. Modifier la valeur par défaut de la colonne `date_fac` de la table `facture`, elle doit maintenant correspondre à la date courante (`current_date`). Écrire la requête qui permet de réaliser cette opération ;

```
ALTER TABLE facture
        ALTER date_fac SET DEFAULT current_date;
```

4. Le diagramme de classes actuel (annexe 1) est incomplet, des modifications sont réalisées en annexe3. Ecrire le script SQL qui permet d'ajouter ces modifications ;

-- Création de la table `type_tel`

```
create table type_tel (
    id_type_tel serial primary key ,
    type_code varchar(8),
    descrip_type varchar(50),
    constraint ck_type_code check(type_code IN('fax', 'tel', 'gsm'))
);
```

```
-- ajout de la clé étrangère dans la table telephone

Alter table telephone
  add id_type_tel int ,
  add constraint fk_teltypetel foreign key (id_type_tel)
                                references type_tel (id_type_tel ) ;

-- Insertion des valeurs dans la table type_tel

Insert into type_tel(type_code, descrip_type)
select distinct typ_code,
       case typ_code when 'gsm'
       then 'portable'
       else typ_code || ' ' || localisation
       end
from telephone
order by typ_code

-- valorisation de la clé étrangère dans la table telephone pour le
  lien vers type_tel
Sous requête corrélées !!!
Update telephone T
Set id_type_tel = (select id_type_tel
                  from type_tel
                  where type_code=T.typ_code
                    and descrip_type= (case T.typ_code when 'gsm'
                    then 'portable'
                    else T.typ_code || ' ' || T.localisation
                    end )
                  )

-- Requête de vérification
  select * from telephone join type_tel using(id_type_tel)

-- suppression des colonnes typ_code et localisation
-- de la table telephone

ALTER TABLE telephone
  DROP COLUMN typ_code,
  DROP COLUMN localisation ;
```

A vertical decorative element on the left side of the slide, composed of a cluster of small hexagons in blue, yellow, and black, arranged in a pattern that tapers towards the top.

Les vues et les règles

A decorative element on the right side of the slide, consisting of a cluster of hexagons in blue, yellow, and black, arranged in a pattern that tapers towards the bottom.

Comment protéger des tables



LES VUES



A decorative vertical pattern on the left side of the slide, composed of various colored hexagons (blue, yellow, black) and dots.

Présentation

■ Introduction

- **Les vues permettent :**
 - **de simplifier le schéma relationnel ;**
 - **D'affiner les privilèges ;**
 - **De cacher certaines colonnes d'une table ;**
 - **Regrouper des informations au sein d'une entité**

Création de vues

■ Syntaxe de création d'une vue

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ]  
VIEW nom [ ( nom de colonne [, ...] ) ]  
AS REQUÊTE
```

- **TEMPORARY ou TEMP** La vue est temporaire., elles sont automatiquement supprimées en fin de session.
- **nom** Le nom de la vue à créer , il ne peut être le même que le nom d'une table
- **nom de colonne** Une liste optionnelle de noms à utiliser pour les colonnes de la vue. Si elle n'est pas donnée, le nom des colonnes est déduit de la requête.
- **requête** Une commande SELECT qui fournira les colonnes et lignes de la vue.

Schéma de données des exemples

chaussure.chaussure

- nom_chaussure : varchar(250)
- # dispo_chaussure : int(11)
- couleur_chaussure : text
- # long_min_chaussure : double
- # long_max_chaussure : double
- unite_nom : varchar(250)

chaussure.lacet

- nom_lacet : varchar(250)
- # dispo_lacet : int(11)
- couleur_lacet : text
- # longueur_lacet : double
- unite_nom : varchar(250)

nom_chaussure	dispo_chaussure	couleur_chaussure	long_min_chaussure	long_max_chaussure	unite_nom
sh1	2	black	70	90	cm
sh2	0	black	30	40	inch
sh3	4	brown	50	65	cm
sh4	3	brown	40	50	inch

chaussure.unite

- unite_nom : varchar(250)
- # facteur_unite : double

unite_nom	facteur_unite
cm	1
m	100
inch	2.54

nom_lacet	dispo_lacet	couleur_lacet	longueur_lacet	unite_nom
sl1	5	black	80	cm
sl2	6	black	100	cm
sl3	0	black	35	inch
sl4	8	black	40	inch
sl5	4	brown	1	m
sl6	0	brown	0.9	m
sl7	7	brown	60	cm
sl8	1	brown	40	inch

Exemple de création de vues

- Création d'une vue qui permet d'avoir les longueurs min et max des chaussures en centimètre

```
CREATE VIEW view_chaussure_cm
AS SELECT nom_chaussure, dispo_chaussure, couleur_chaussure,
        long_min_chaussure, long_max_chaussure, s.unite_nom,
        long_min_chaussure * facteur_unite AS long_min_chaussure_cm,
        long_max_chaussure * facteur_unite AS long_max_chaussure_cm
FROM chaussure s JOIN unite u ON s.unite_nom = u.unite_nom;
```

```
SELECT *
FROM view_chaussure_cm ;
```

nom_chaussure	dispo_chaussure	couleur_chaussure	long_min_chaussure	long_max_chaussure	unite_nom	long_min_chaussure_cm	long_max_chaussure_cm
sh1	2	black	70	90	cm	70	90
sh2	0	black	30	40	inch	76.2	101.6
sh3	4	brown	50	65	cm	50	65
sh4	3	brown	40	50	inch	101.6	127

Exemple de création de vues

- La même mais en changeant le nom des colonnes

ATTENTION impossible de remplacer une vue existante en lui changeant le nom des colonnes

```
CREATE OR REPLACE VIEW view_chaussure_cm (nom, stock, couleur, long_min, long_max,  
                                         unite,long_min_cm,long_max_cm)  
AS SELECT nom_chaussure, dispo_chaussure, couleur_chaussure,  
           long_min_chaussure, long_max_chaussure, s.unite_nom,  
           long_min_chaussure * facteur_unite AS long_min_chaussure_cm,  
           long_max_chaussure * facteur_unite AS long_max_chaussure_cm  
FROM chaussure s JOIN unite u ON s.unite_nom = u.unite_nom;
```

```
SELECT *  
FROM view_chaussure_cm ;
```

nom	stock	couleur	long_min	long_max	unite	long_min_cm	long_max_cm
sh1	2	black	70	90	cm	70	90
sh2	0	black	30	40	inch	76.2	101.6
sh3	4	brown	50	65	cm	50	65
sh4	3	brown	40	50	inch	101.6	127

Utilisation d'une vue

■ L'interrogation de données

- Une vue peut être interrogée via une requête **SELECT** au même titre que les tables

- Exemple

- Requête simple

```
SELECT nom, stock, couleur  
FROM view_chaussure_cm  
WHERE long_max_cm <=90 ;
```

nom	stock	couleur
sh1	2	black
sh3	4	brown

Utilisation d'une vue

■ L'interrogation de données

■ Exemple

■ Requête avec jointure

```
SELECT nom,stock,couleur,unite, facteur_unite  
FROM view_chaussure_cm JOIN unite ON unite=unite_nom  
WHERE long_max_cm <=90 ;
```

nom	stock	couleur	unite	facteur_unite
sh1	2	black	cm	1
sh3	4	brown	cm	1

Utilisation d'une vue

- L'interrogation de données
 - Exemple
 - Requête avec sous requête

```
SELECT nom, stock, couleur, unite
FROM view_chaussure_cm Vc
WHERE exists ( SELECT *
                FROM lacet
                WHERE couleur_lacet= Vc.couleur) ;
```

nom	stock	couleur	unite
sh1	2	black	cm
sh2	0	black	inch
sh3	4	brown	cm
sh4	3	brown	inch

Utilisation d'une vue

- Mise à jours, suppression et insertion de données
 - **les vues sont en lecture seule : le système n'autorise pas une insertion, une mise à jour ou une suppression sur une vue.**
 - **Si l'on veut actualiser (insert, update ou delete) des données des tables à travers les vues il faut créer des règles (RULES) sur les vues .**



Utilisation d'une vue

- **Mise à jours, suppression et insertion de données**
 - **Sans ce mécanisme de règles (RULES) sur les vues, si l'on tente de d'actualiser une vue on aura l'erreur suivante :**

```
UPDATE view_chaussure_cm  
SET stock = 13  
WHERE nom = 'sh4';
```

Erreur SQL :

```
ERREUR: ne peut pas mettre à jour la vue « view_chaussure_cm »  
HINT: Vous avez besoin d'une règle non conditionnelle ON UPDATE DO INSTEAD ou d'un trigger INSTEAD OF UPDATE.
```

Suppression d'une vue

- Supprimer une vue

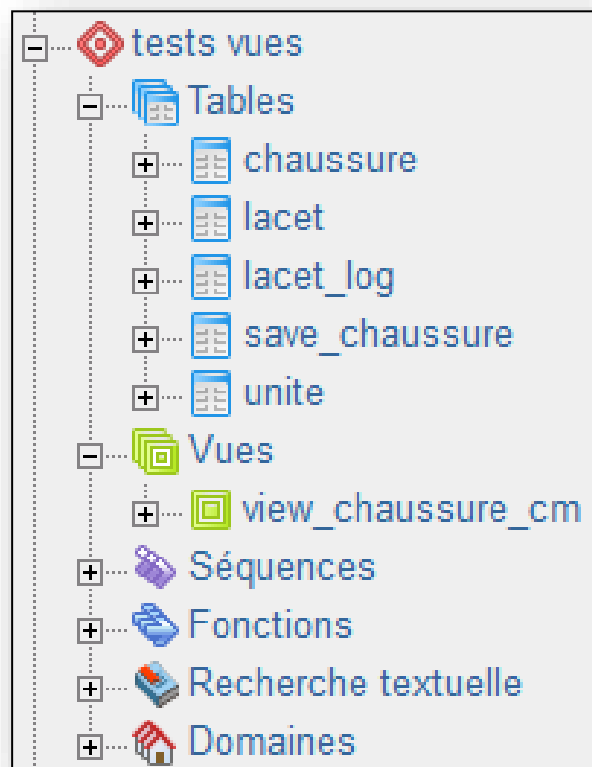
```
DROP VIEW VIEW_NAME [, VIEW_NAME] ...
```

Exemple :

```
DROP VIEW view_chaussure_cm;
```

Les vues sous phppgadmin

- Une vue est un objet situé au même niveau que les tables



A vertical decorative element on the left side of the slide, composed of a cluster of small blue, yellow, and black hexagons.

LES RÈGLES

A decorative element on the right side of the slide, consisting of a cluster of blue, yellow, and black hexagons, some of which are larger and more prominent than others.

A decorative graphic on the left side of the slide, consisting of a vertical arrangement of blue and yellow hexagons of various sizes, some solid and some outlined.

Les règles

■ Présentation

- **Le système de règles autorise la définition d'actions alternatives sur les insertions, mises à jour ou suppressions dans les tables ou les vues.**
- **Une règle permet d'imposer des commandes supplémentaires, de remplacer les commandes par d'autres, voire d'empêcher sa réalisation lors de l'exécution d'une instruction sur une table ou une vue donnée.**
- **Une règle est déclenchée sur un des événements INSERT, UPDATE ou DELETE**

Syntaxe de création d'une règle

```
CREATE [ OR REPLACE ] RULE nom  
AS ON événement TO table ou vue [ WHERE condition ]  
DO [ ALSO | INSTEAD ]  
{commande | (commande; commande..) }
```

nom : Le nom de la règle à créer.

événement : SELECT, INSERT, UPDATE ou DELETE.

condition : Toute expression SQL conditionnelle (renvoyant un type boolean). L'expression de la condition ne peut pas faire référence à une table autre que **NEW** ou **OLD** ni contenir de fonction d'agrégat.

INSTEAD : Les commandes sont exécutées à la place de la commande originale.

ALSO : Les commandes sont exécutées *en plus de* la commande originale (par défaut).

Les structures OLD et NEW

- Elles sont automatiquement créées par le système sur une action de type UPDATE, DELETE ou INSERT sur une table, lors du déclenchement d'une règle ou d'un trigger.
- Elles ont la structure de la table sur laquelle à lieu l'action
- La structure OLD
 - Existe après un UPDATE ou un DELETE
 - Contient la ligne qui a été supprimé ou celle avant modification
- La structure NEW
 - Existe après un UPDATE ou un INSERT
 - Contient la ligne qui a ajouter ou celle après modification

Les structures OLD et NEW

■ Exemple sur un UPDATE

```
UPDATE chaussure
SET  dispo_chaussure = 12,
     couleur_chaussure = 'Red'
WHERE nom_chaussure = 'sh1';
```

table avant le update

nom_chaussure	dispo_chaussure	couleur_chaussure
sh1	2	black
sh2	0	black
sh3	4	brown
sh4	3	brown

■ Pour accéder à la colonne d'une des structures OLD ou NEW

OLD.dispo_chaussure contient 2
OLD.couleur_chaussure contient *black*
et
NEW.dispo_chaussure contient 12
NEW.couleur_chaussure contient *Red*

Les structures OLD et NEW

■ Exemple sur un INSERT

```
INSERT INTO chaussure values ('sh5', 12, 'red', 45, 69, 'cm');
```

nom_chaussure	dispo_chaussure	couleur_chaussure	long_min_chaussure	long_max_chaussure	unite_nom
sh1	2	black	70	90	cm
sh2	0	black	30	40	inch
sh3	4	brown	50	65	cm
sh4	3	brown	40	50	inch
sh5	12	red	45	69	cm

■ Pour accéder à la colonne de la structure **NEW**

NEW.dispo_chaussure contient 12

NEW.couleur_chaussure contient Red

NEW.nom_chaussure contient sh5

.....

La structure OLD n'existe pas sur un INSERT

Les structures OLD et NEW

■ Exemple sur un DELETE

```
DELETE FROM chaussure  
WHERE nom_chaussure = 'sh5';
```

■ Pour accéder à la colonne de la structure OLD

```
OLD.dispo_chaussure contient 12  
OLD.couleur_chaussure contient Red  
OLD.nom_chaussure contient sh5  
.....
```

La structure NEW n'existe pas sur un DELETE

Exemple de règle sur une table

■ Exemple 1 : ajout d'action avec condition

Nous voulons tracer les modifications dans la colonne *dispo_lacet* de la table *lacet*.

Créons une table de log et une règle qui va écrire une entrée lorsqu'un *update* est lancé sur la table *lacet*.

La table lacet_log

```
chaussure.lacet_log
nom_lacet : varchar(250)
# dispo_lacet : int(11)
log_who : varchar(250)
log_when : timestamp
```

Exemple de règle sur une table

■ Exemple 1 : ajout d'action avec condition

Création de la règle sur la table lacet

```
CREATE RULE log_lacet AS ON UPDATE TO lacet
WHERE NEW.dispo_lacet <> OLD.dispo_lacet
DO
INSERT INTO lacet_log
VALUES (NEW.nom_lacet,NEW.dispo_lacet,current_user,current_timestamp);
```

La règle *log_lacet* sera déclenchée sur un **UPDATE** sur la table *lacet*, uniquement si la valeur de la colonne *dispo_lacet* a changé, alors en plus de l'action sur la table *lacet* on ajoute une ligne dans la table des logs (*lacet_log*)

Exemple de règle sur une table

■ Exemple 1 : ajout d'action avec condition

Mise à jour sur la table lacet

```
UPDATE lacet
SET dispo_lacet = 12,
    couleur_lacet = 'Red',
    longueur_lacet=50
WHERE nom_lacet = 'sl8';
```

nom_lacet	dispo_lacet	couleur_lacet	longueur_lacet	unite_nom
sl1	5	black	80	cm
sl2	6	black	100	cm
sl3	0	black	35	inch
sl4	8	black	40	inch
sl5	4	brown	1	m
sl6	0	brown	0.9	m
sl7	7	brown	60	cm
sl8	12	Red	50	inch

Le update est appliqué sur la table *lacet* et une ligne a été ajouté dans la table *lacet_logs*

nom_lacet	dispo_lacet	log_who	log_when
sl8	12	postgres	2014-03-20 18:16:11.158

Exemple de règle sur une table

■ Exemple 1 : ajout d'action avec condition

On peut mettre autant d'action que nécessaire dans la règle

```
CREATE RULE log_lacet AS ON UPDATE TO lacet  
WHERE NEW.dispo_lacet <> OLD.dispo_lacet  
DO
```

```
( INSERT INTO lacet_log  
    VALUES ( NEW.nom_lacet, NEW.dispo_lacet,  
              current_user, current_timestamp );  
SELECT * FROM lacet_log;  
)
```

Exemple de règle sur une table

■ Exemple 2 : remplacer les actions de mises à jour avec condition

Nous allons dérouter l'action d'insertion si la valeur de la clé primaire existe dans la table *unite*.

Dans ce cas nous ferons une mise à jour sur la ligne de la table *unite*

```
CREATE RULE insert_unite AS ON INSERT TO unite
WHERE exists ( SELECT * FROM unite
                WHERE unite_nom=NEW.unite_nom)
DO INSTEAD
  UPDATE unite
    SET facteur_unite=NEW.facteur_unite
    WHERE unite_nom=NEW.unite_nom
```

Exemple de règle sur une table

- Exemple 2 : remplacer les actions de mises à jour avec condition

Insertion d'une donnée valide :

unite_nom	facteur_unite
cm	1
m	100
inch	2.54

```
INSERT INTO unite VALUES('mm',0.01);
```

La règle n'est pas déclenchée l'insertion se déroule normalement

unite_nom	facteur_unite
cm	1
m	100
inch	2.54
mm	0.01

Exemple de règle sur une table

- Exemple 2 : remplacer les actions de mises à jour avec condition
Insertion d'une donnée avec doublon sur la clé primaire :

```
INSERT INTO unite VALUES('mm',0.1);
```

unite_nom	facteur_unite
cm	1
m	100
inch	2.54
mm	0.01

*La règle est déclenchée, l'insertion n'a pas lieu,
mais la règle s'applique et la mise à jour de l'unite
prend la place de l'insertion*

unite_nom	facteur_unite
cm	1
m	100
inch	2.54
mm	0.1

Création d'une règle sur une vue

■ Limitation des règles sur les vues

- l'utilisation de **règles conditionnelles** pour la mise à jour de vues **n'est pas possible**.
- Chaque action autorisée sur la vue *doit* correspondre une règle INSTEAD inconditionnelle.
- Si la règle est conditionnelle ou n'est pas une règle INSTEAD, alors le système rejette toute tentative de mise à jour.

Création d'une règle sur une vue

- **Limitation des règles sur les vues**

- Exemple à ne pas faire

```
CREATE or replace RULE update_view_chaussure
AS ON UPDATE TO view_chaussure_cm
WHERE NEW.nom = OLD.nom
DO
    UPDATE chaussure
    SET dispo_chaussure = new.stock,
        long_min_chaussure = new.long_min,
        long_max_chaussure = new.long_max;
```

Création d'une règle sur une vue

■ Limitation des règles sur les vues

■ Exemple à ne pas faire

```
UPDATE view_chaussure_cm  
SET stock = 21,  
    long_min = 60  
WHERE nom = 'sh6';
```

Suite à la définition de la règle précédente cette requête générera l'erreur suivante :

Erreur SQL :

```
ERREUR: ne peut pas mettre à jour la vue « view_chaussure_cm »  
HINT: Vous avez besoin d'une règle non conditionnelle ON UPDATE DO INSTEAD ou d'un trigger INSTEAD OF UPDATE.
```

Création d'une règle sur une vue

- **Limitation des règles sur les vues**
 - **Exemple à ne pas faire**

```
CREATE or replace RULE update_view_chaussure
AS ON UPDATE TO view_chaussure_cm
WHERE NEW.nom = OLD.nom
DO
    UPDATE chaussure
    SET dispo_chaussure = new.stock,
        long_min_chaussure = new.long_min,
        long_max_chaussure = new.long_max;
```

Pas de condition et doit
être obligatoirement
être **INSTEAD**

Création d'une règle sur une vue

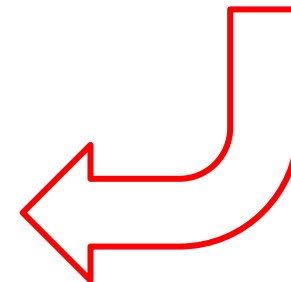
■ Limitation des règles sur les vues

■ Exemple à ne pas faire

Doit être remplacé par :

```
CREATE or replace RULE update_view_chaussure
AS ON UPDATE TO view_chaussure_cm
DO INSTEAD
  UPDATE chaussure
  SET dispo_chaussure = NEW.stock,
      long_min_chaussure = NEW.long_min,
      long_max_chaussure = NEW.long_max
  WHERE nom_chaussure = OLD.nom;
```

```
Update view_chaussure_cm
Set stock=23
    long_max=56
Where nom='sh4'
```



Attention avec la structure OD Let NEW

```
CREATE or replace RULE update_view_chaussure
```

```
AS ON delete TO unite
```

```
DO INSTEAD
```

```
(
```

```
delete FROM chaussure
```

```
WHERE nom_unite= OLD.nom_unite; fait référence à la table unite
```

```
delete FROM lacet
```

```
WHERE nom_unite= OLD.nom_unite; fait référence à la table chaussure
```

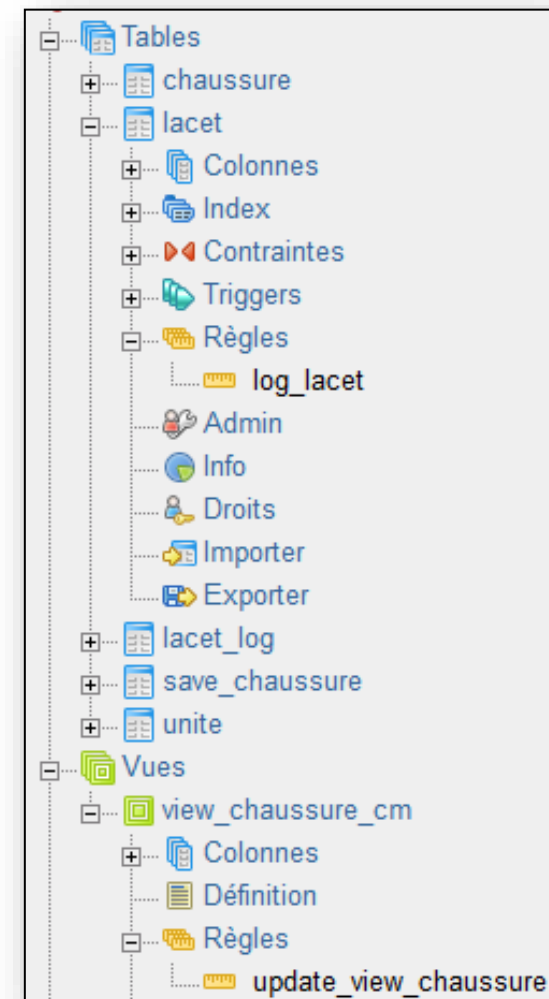
```
delete FROM unite
```

```
WHERE nom_unite= OLD.nom_unite; fait référence à la table lacet
```

```
)
```

Les règles sous phppgadmin

- Les règles sont des objets contenu dans les tables ou les vues, en effet les règles s'appliquent à une et seule vue ou table



TRAVAIL A FAIRE

Q1. CREATION DE VUES

A. Vue clients et gites

Créer une vue, que vous nommerez *view_location*, qui permet de voir les clients et les gites qu'ils ont loués, afficher le résultat de la vue créée.

```
CREATE OR REPLACE VIEW view_location (numcli, nom_cl, duree_loc, nbper, date_loc, numgite, nomgite)
```

```
AS
```

```
select C.numcli,nomcli,duree,nbper,date_debut,G.numgite,nomgite
from clients C join louer L on C.numcli=L.numcli
      join gites G on G.numgite=L.numgite
order by C.numcli;
```

```
-- affichage du résultat de la vue
```

```
select *
from view_location;
```

numcli	nom_cl	duree_loc	nbper	date_loc	numgite	nomgite
1	Pruitt	7	2	2011-01-19	1	Le Castor
2	Anderson	13	11	2011-09-27	3	Belle pente
2	Anderson	8	4	2013-06-27	3	Belle pente
3	Perry	11	2	2011-09-15	4	La belle bosse
6	Ramirez	7	7	2011-05-29	5	l'Edelweiss
7	Huff	6	12	2014-09-15	4	La belle bosse
7	Huff	12	7	2014-03-06	4	La belle bosse

44 lignes

B. Vue gites, propriétaire et tarif

Créer une vue, que vous nommerez *view_tarif_gite*, qui permet de voir les gites, leur propriétaire et les tarifs en fonction de la saison, afficher le résultat de la vue créée

```
CREATE OR REPLACE VIEW view_tarif_gite (numgite, nom_gite, nom_proprietaire, saison, tarif)
```

```
AS
```

```
select G.Numgite,nomgite,nomprop, saison, tarifttc
from gites G join facturer F on G.numgite=F.numgite
      join proprietaire P ON P.numprop=G.numprop
order by g.numgite;
```

```
-- affichage du résultat de la vue
```

```
select *
from view_tarif_gite;
```

numgite	nom_gite	nom_proprietaire	saison	tarif
1	Le Castor	Finn	HAUTE	600.00
1	Le Castor	Finn	BASSE	387.00
1	Le Castor	Finn	MOYENNE	450.00
2	Le col blanc	Clinton	BASSE	300.00
2	Le col blanc	Clinton	HAUTE	700.00
2	Le col blanc	Clinton	MOYENNE	400.00

51 lignes

C. Vue gites jamais loués

Créer une vue, que vous nommerez *view_gite_sans_loc*, qui permet de voir les gites jamais loués, afficher le résultat de la vue créée

```
CREATE OR REPLACE VIEW view_gite_sans_loc (num_gite, nom_gite, nom_proprietaire,
saison, tarif, tel_gite, description,num_prop, superficie_gite,nb_epis,nb_personne)
AS
```

```
SELECT numgite, nomgite,nomprop, saison, tarifttc,telgite, description,
numprop, superficiegit, nbepis, nbpers
FROM gites JOIN proprietaire using(numprop)
JOIN facturer using(numgite)
WHERE numgite NOT IN(SELECT numgite FROM louer);
```

```
-- affichage du résultat de la vue
select *
from view_gite_sans_loc;
```

num_gite	nom_gite	nom_proprietaire	saison	tarif	tel_gite	description	num_prop	superficie_gite	nb_epis	nb_personne
2	Le col blanc Clinton		BASSE	300.00	07-84-59-55-17	Lorem ipsum dolor	14	NULL	2	10
2	Le col blanc Clinton		HAUTE	700.00	07-84-59-55-17	Lorem ipsum dolor	14	NULL	2	10
2	Le col blanc Clinton		MOYENNE	400.00	07-84-59-55-17	Lorem ipsum dolor	14	NULL	2	10
6	Nevada	Quamar	HAUTE	800.00	07-47-05-64-09	Lorem ipsum dolor sit amet, consectetur adipisci...	11	NULL	1	9
6	Nevada	Quamar	BASSE	450.00	07-47-05-64-09	Lorem ipsum dolor sit amet, consectetur adipisci...	11	NULL	1	9
6	Nevada	Quamar	MOYENNE	600.00	07-47-05-64-09	Lorem ipsum dolor sit amet, consectetur adipisci...	11	NULL	1	9
11	Basil	Carly	HAUTE	1355.00	05-74-51-48-45	Lorem ipsum dolor	7	NULL	1	5
11	Basil	Carly	BASSE	850.00	05-74-51-48-45	Lorem ipsum dolor	7	NULL	1	5
11	Basil	Carly	MOYENNE	1050.00	05-74-51-48-45	Lorem ipsum dolor	7	NULL	1	5
17	la camelle	Chancellor	BASSE	350.00	06-83-42-07-43	Lorem ipsum dolor sit amet, consectetur	17	NULL	1	13
17	la camelle	Chancellor	HAUTE	750.00	06-83-42-07-43	Lorem ipsum dolor sit amet, consectetur	17	NULL	1	13
17	la camelle	Chancellor	MOYENNE	550.00	06-83-42-07-43	Lorem ipsum dolor sit amet, consectetur	17	NULL	1	13

D. Vue gites et activités

Créer une vues qui permet de voir les gites et les activités associées
Attention à la colonne info de la table *proposer*

```
CREATE OR REPLACE VIEW view_gite_activite (numgite, nom_gite,
activite,distance,interet)
AS
select G.numgite,nomgite,nomact,(info).distance, (info).interet
from gites G join proposer P on P.numgite=G.numgite
join activites A on A.numact=P.numact;
```

```
-- affichage du résultat de la vue
select *
from view_gite_activite;
```

numgite	nom_gite	activite	distance	interet
13	Germaine	Escalade	13	5
9	Maris	alpinisme	9	1
13	Germaine	Randonnée	13	2
12	le half-pipe	Ski	12	3
9	Maris	Canyoning	9	4
11	Basil	Randonnée	11	2
4	La belle bosse	alpinisme	4	1

29 lignes

Q2. CREATION DE REGLES

A. règle sur un update

Créer une règle sur la vue view_location qui lors de la mise à jours de la durée de location (duree_loc) et/ou le nombre de personnes (nbper) met à jours la table correspondante (louer), *La mise à jour ne se fera que si le nombre de personnes est inférieur ou égal au nombre de personnes défini dans le gite*
testez cette règle

```
CREATE OR REPLACE RULE update_view_location
AS ON UPDATE TO view_location
DO INSTEAD
  UPDATE louer
  SET duree = NEW.duree_loc,
      nbper = NEW.nbper
  WHERE numgite = NEW.numgite
      AND numcli = NEW.numcli
      AND date_debut=NEW.date_loc
      AND NEW.nbper <=(SELECT nbpers FROM gites WHERE numgite= NEW.numgite);

-- test
update view_location
set duree_loc=3,
    nbper=3
where numgite=1 and numcli=1 and date_loc='2011-01-19';
-- mise à jour possible nbper <= 4 : 1 ligne mises à jours

update view_location
set nbper=2
where numgite=1;
-- Les autres critères ne sont pas pris en compte : 4 lignes mises à jours

update view_location
set nbper=9
where numgite=1 ;
-- pas de mise à jours nbper > le nb pers du gite 1
```

B. règle sur un delete

Lors de la suppression d'un gite via la vue view_location, il faut supprimer les lignes des différentes tables qui font référence au gite à supprimer !!

```
CREATE or replace RULE delete_view_location
AS ON delete TO view_location
DO INSTEAD
(
  DELETE from chambres WHERE numgite = OLD.numgite;

  DELETE from facturer WHERE numgite = OLD.numgite;

  DELETE from proposer_par WHERE numgite = OLD.numgite;

  DELETE from proposer WHERE numgite = OLD.numgite;

  DELETE from louer WHERE numgite = OLD.numgite;

  DELETE from gites WHERE numgite = OLD.numgite; );
```

```
-- test de la règle
DELETE FROM view_location
WHERE numgite=1;

SELECT *
FROM gites NATURAL JOIN louer
        NATURAL JOIN chambres
        NATURAL JOIN proposer
WHERE numgite=1 ;

-- Pas de ligne retournée par le select
Par contre le gîte existe encore ???

SELECT *
FROM gites
WHERE numgite=1 ;
```

C. Règle sur un insert

*Ajouter une règle pour éviter une erreur lors de l'insertion si doublon sur une pk.
lors de l'insertion d'un nouveau gîte (table gites) si la valeur de la clé primaire existe il faudra, à la place de
l'insertion, mettre à jour le gîte avec les données de l'insertion..*

```
CREATE OR REPLACE RULE insert_gite AS ON INSERT TO gites
WHERE exists (SELECT * FROM gites WHERE numgite=NEW.numgite)
DO INSTEAD
UPDATE gites
SET nomgite=NEW.nomgite,
adrgite=NEW.adrgite,
cpgite=NEW.cpgite,
villegite=NEW.villegite,
telgite=NEW.telgite,
description=NEW.description,
numprop=NEW.numprop,
superficiegit=NEW.superficiegit,
nbepis=NEW.nbepis,
nbpers=NEW.nbpers
WHERE numgite=NEW.numgite;

-- test de la règle avec ajout effectif
INSERT INTO gites VALUES(default, 'les castors', '4 rue des castors', 73450,
                          'Valmorel', '06-06-07-08-09', 'bien', 16, 120, 3, 5) ;

-- test de la règle avec modification de la ligne du gîte N°13
INSERT INTO gites VALUES(13, 'Germaine', 'CP 344, 5737 Scelerisque Avenue',
                          73607, 'albertville', '04-66-88-83-44', 'Lorem', 1, 150, 2, 6);
```

Q3. MAINTENANCE CORRECTIVE DE LA BDD

Il s'agit de vérifier que le nombre de personnes déclarés par gîte (nbpers) est bien conforme au nombre de couchages, lits simple et double (nblits, nblitdo)

- Écrire une requête qui permet d'afficher :
 - le N° du gîte
 - Le nombre de personnes déclaré dans le gîte
 - Le nombre de lits simple et le nombre lits double
 - et la capacité de couchage par gîte, en fonction du nombre de lit et double (1 place par lit simple et 2 par lit double)

```
SELECT      G.numgite as "N° du gite",
            SUM (nblits) AS "nb lits simple",
            SUM (nblitdo) AS "nb lits double",
            SUM (nblits*1+ nblitdo*2) AS "capacité de couchage",
nbpers AS "Nb personnes déclaré",
FROM gites G JOIN chambres C ON G.numgite=C.numgite
GROUP BY G.numgite
HAVING nbpers!= SUM(nblits*1+ nblitdo*2)
ORDER BY 1;
```

N° du gîte	Nb personnes déclaré	nb lits simple	nb lits double	capacité de couchage
7	28	13	7	27
9	10	4	4	12
11	5	3	2	7
12	2	3	1	5
13	6	3	1	5

- Écrire une requête qui permet de mettre en corrélation le nombre de personnes déclaré avec la capacité de couchage (une requête corrélée et nécessaire pour s'en sortir)
Exemple : Si la capacité de couchage est de 6 il faut que le nombre de personne de la table *gites* soit aussi à 6

```
update gites Gi
set nbpers=(SELECT sum(nblits*1+ nblitdo*2)
            FROM chambres where numgite=Gi.numgite)
WHERE nbpers!= (SELECT sum(nblits*1+ nblitdo*2)
               FROM chambres where numgite=Gi.numgite) ;
```

1. Les vues

1. Créez une vue nommée pts_GP_Malaisie

```
CREATE OR REPLACE VIEW pts_GP_Malaisie(prenom, nom, Position_Arrivee, Pts_Asquis)
AS
SELECT nompilote, prenompilote, positionarrivee,
coalesce((select points from bareme where place=positionarrivee::int),0)
FROM grandprix G JOIN courir C ON C.idgp=G.idgp
        JOIN pilote P ON P.idpilote=C.idpilote
WHERE nomgp='Grand Prix de Malaisie'
AND positionarrivee !='A'
AND positionarrivee !='0' ;

-- test le de la vue : SELECT * FROM
pts_GP_Malaisie;
```

prenom	nom	position_arrivee	pts_asquis
Rosberg	Nico	1	25
Kvyat	Daniil	2	18
Bottas	Valtteri	3	15
Nasr	Felipe	4	12
Pérez	Sergio	5	10
Hülkenberg	Nico	6	8
Massa	Felipe	7	6
Ricciardo	Daniel	8	4
Hamilton	Lewis	9	2
Gutiérrez	Esteban	10	1
Alonso	Fernando	11	0
Button	Jenson	12	0
Grosjean	Romain	13	0
Palmer	Jolyon	14	0
Verstappen	Max	16	0
Räikkönen	Kimi	17	0
Ericsson	Marcus	18	0
Vettel	Sebastian	19	0
Wehrlein	Pascal	20	0
Sainz Jr.	Carlos	21	0

2. Créez une vue nommée nb_km_GP,

```
CREATE OR REPLACE VIEW nb_km_GP(nomGP, longueurGP) AS
SELECT nomgp, nbtour*longpiste
FROM grandprix G JOIN circuit C ON C.id_circuit=G.id_circuit ;

-- test le de la vue :
SELECT * FROM nb_km_GP
```

nomgp	longueurgp
Grand Prix d'Australie	577.20
Grand Prix des Etats-Unis	1070.88
Grand Prix de Bahrein	1330.86
Grand Prix d'Espagne	1145.40
Grand Prix du Canada	819.00
Grand Prix d'Allemagne	1335.00
Grand Prix de Hongrie	921.20
Grand Prix du Brésil	1190.40
Grand Prix de Monaco	846.80
Grand Prix d'Italie	1905.00
Grand Prix d'Autriche	2214.80
Grand Prix de Malaisie	1009.92
Grand Prix de Chine	3069.40
Grand Prix de Grande-Bretagne	1470.00
Grand Prix de Singapour	834.60
Grand Prix de Belgique	2440.00
Grand Prix de Russie	3064.40
Grand Prix du Japon	1405.80
Grand Prix d'Abou Dabi	765.60

19 ligne(s)

3. Créez une vue nommée `vue_plus_de_sponsor`,

```
CREATE OR REPLACE VIEW vue_plus_de_sponsor( "Prenom pilote", "Nom pilote") AS
select prenompilote, nompilote from soutenir S join pilote P ON
P.idpilote=S.idpilote group by nompilote, P.idpilote
having count(*) >= ALL(select count(*) from soutenir group by idpilote);

-- test de la vue
select *
from vue_plus_de_sponsor
```

Prenom pilote	Nom pilote
Nico	Rosberg
Lewis	Hamilton

2. Les règles

1. Règle sur un update sur la vue `view_position_arr_pilote`

2.1.a / affiche le résultat de la vue

```
SELECT * FROM view_position_arr_pilote;
```

2.1.b /

```
CREATE or replace rule regle_update_view_position_arr_pilote
AS ON UPDATE TO view_position_arr_pilote
DO INSTEAD
(
    -- modification du nom et/ou prenom du pilote
    UPDATE pilote
    SET nompilote = NEW.nom,
    prenompilote = NEW.prenom
    WHERE idpilote = OLD.id_pilote;

    -- modification du nom et/ou prenom du pilote
    UPDATE courir
    SET positionarrivee = NEW.position
    WHERE idpilote = OLD.id_pilote AND idgp=OLD.id_gp;
);
```

Test de la règle

```
UPDATE view_position_arr_pilote
SET nom='DURAND',
    prenom='Carlos', position=20
WHERE id_pilote=18 AND id_gp=19 ;

UPDATE view_position_arr_pilote
SET position=1
WHERE id_pilote=18 AND id_gp=19
```

2. Règle sur un insert sur la table circuit

```
CREATE RULE regle_insert_circuit AS ON INSERT TO circuit
WHERE exists (SELECT * FROM circuit WHERE id_circuit=NEW.id_circuit)
DO INSTEAD
  UPDATE circuit
  SET nomcircuit =NEW.nomcircuit,
      Payscircuit =NEW.payscircuit,
      Longpiste   =NEW.longpiste,
      Nbspectateur =NEW.nbspectateur
  WHERE id_circui =NEW.id_circuit;
```

-- insertion effective SANS déclenchement de la règle

```
INSERT INTO circuit VALUES (20, 'le miens', 'France', 5, 6700);
```

-- insertion effective AVEC déclenchement de la règle

```
INSERT INTO circuit VALUES (20, 'Le castelet', 'France', 5, 34000);
```

Le Langage SQL

Parcours d'arbre avec la récursivité

Les CTE "Common Table Expression"

- Les CTE "Common Table Expression" (ou *Table d'Expression Partagées* en français) permettent de créer des vues (vue au sens SGBD) mais locales, c'est à dire utilisables uniquement dans la requête en cours.
- Une utilisation de cette fonctionnalité est de découper des requêtes complexes en parties plus simples:

WITH

sous_requete1 AS (SELECT FROM),

sous_requete2 AS (SELECT FROM)

SELECT * FROM ma_table, sous_requete1, sous_requete2 ;

Les requêtes qui génèrent
les vues locales

La requête qui utilise les vues locales

Exemple de CTE

WITH

```
R1 AS (SELECT cli_id,nom
        FROM client WHERE enseigne is NULL),
R2 AS (SELECT cli_id,date_jour,nb_personne
        FROM reserve
        where date_jour = '2000-03-03')
```

```
SELECT *
FROM client
where cli_id in(select distinct cli_id from R2)
and cli_id in(select distinct cli_id from R1);
```

Les requêtes qui génèrent
les vues locales (R1 et R2)

La requête qui utilise
les vues locales (R1 et R2)

Autre Exemple de CTE

WITH

T1 (nom, nb_ade) AS

(

SELECT nom_etu, COUNT(DISTINCT adresse)

FROM Etudiant

GROUP BY nom_etu

),

T2 (nom, nb_ads) AS

(

SELECT P.nom_pers, COUNT(DISTINCT adresse)

FROM Personnel P INNER JOIN Societe S ON P.ids = S.ids

GROUP BY P.nom_pers

)

SELECT T1.nom, nb_ade, nb_ads

FROM T1 join T2 ON T1.nom = T2.nom;

On veut afficher le nombre d'adresse par étudiant (nb_ade) et le nombre d'adresse par personnel (nb_ads)

	ABC nom ▼	123 nb_ade ▼	123 nb_ads ▼
1	Dupond	2	1
2	Durand	1	1
3	Duvent	1	1

Autre Exemple de CTE

- Recherche de l'étudiant qui a la meilleur moyenne

```
WITH T1 (nom, moyenne) AS
(
    SELECT nom_etu, AVG(note)
    FROM Etudiant JOIN convention USING(ide)
    GROUP BY ide
),
T2 (maxi) AS
(
    SELECT max(moyenne)
    FROM T1
)
SELECT * from T1
WHERE moyenne=(select maxi from T2)
```

	ABC nom ▼	123 moyenne ▼
1	Dupond	17

A decorative graphic on the left side of the slide consists of a vertical column of colored dots (blue, yellow, black) and hexagons (blue, yellow) of various sizes.

La récursivité en SQL

- Néanmoins, la syntaxe **WITH**, bien que déjà très utile, ne se limite pas à ça.
- La norme **SQL:1999** a permis de transposer en SQL la récursivité en étendant la syntaxe **WITH**.
- Concrètement, en ajoutant le mot clé **RECURSIVE**, on peut dans l'expression d'une CTE faire appel à elle même.

La récursivité en SQL

- Pour mieux comprendre voici un exemple :

```
WITH RECURSIVE t(nombre) AS (
```

```
VALUES(2)
```

Valeur de départ

```
UNION ALL
```

```
SELECT 2 * nombre
```

Calcul avec le nombre courant

```
FROM t
```

```
WHERE 2 * nombre < 100
```

Condition pour rester dans la
récursion

```
) SELECT nombre FROM t;
```



nombre
2
4
8
16
32
64

Cet exemple génère la suite des puissances de 2 inférieures à 100.

La récursivité en SQL

- Une requête récursive s'écrit donc ainsi :

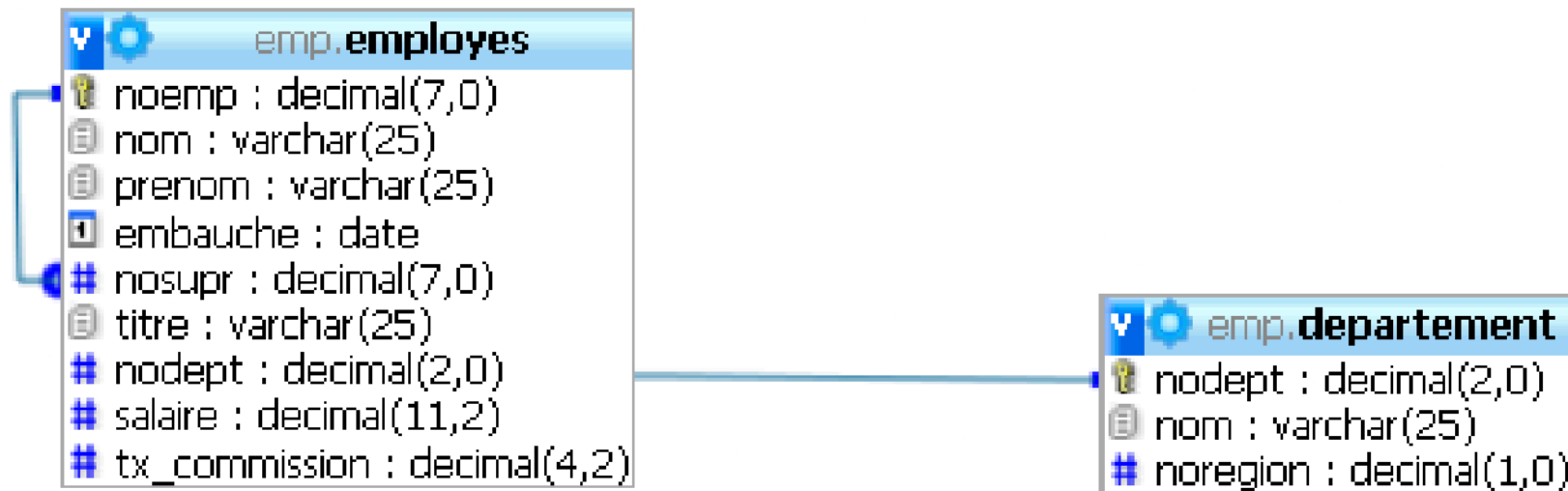
```
WITH RECURSIVE RECURSION (liste_champ) AS(  
    SELECT liste_champ  
    FROM table  
    UNION ALL  
    SELECT liste_champ  
    FROM RECURSION JOIN table ON critère de jointure  
    WHERE condition pour rester  
) SELECT * FROM RECURSION;
```

Attention la liste des champs
doit être la même dans chaque
requête de l'union et la CTE

- On fait donc une CTE dans laquelle on a une union, la deuxième partie de l'UNION faisant appel à la CTE elle même.
- Attention aux requêtes infinies !!!! votre requête doit impérativement être écrite de manière à éviter toute récursion infinie.

Exemple de récursivité

- Voyons avec un exemple un peu plus pratique. Prenons le schéma de données suivant :



- Dans la table *employees*, le champ *nosupr* fait référence au supérieur direct de l'employé qui est lui-même un employé. L'utilisation de requêtes récursives va ici nous permettre des choses très pratiques :

Exemple de récursivité

noemp	nom	prenom	embauche	nosupr	titre	nodept	salaire	tx_commission
1	Potamob	Adhémar	2000-03-28	NULL	President	50	50000.00	NULL
2	Leublouze	Agothe	2000-04-15	1	Dir. Distrib	41	35000.00	NULL
3	Kuzbidon	Alex	2000-05-05	1	Dir. Vente	31	34000.00	NULL
4	Locale	Anasthasie	2000-05-25	1	Dir. Finance	10	38000.00	NULL
5	Teutmaronne	Armand	2000-06-14	1	Dir. Administr	50	38000.00	NULL
6	Joudankou	Debbie	2000-07-04	2	Chef Entrepôt	41	25000.00	NULL
7	Rivenbusse	Elsa	2000-07-24	2	Chef Entrepôt	42	24000.00	NULL
8	Ardelpic	Helmut	2000-08-13	2	Chef Entrepôt	43	23000.00	NULL
9	Peursoonla	Humphrey	2000-09-02	2	Chef Entrepôt	44	22000.00	NULL
10	Vrante	Hélène	2000-09-22	2	Chef Entrepôt	45	21000.00	NULL
11	Enfaillite	Mélusine	2000-10-12	3	Représentant	31	25000.00	10.00
12	Eurktumeme	Odile	2000-11-01	3	Représentant	32	28000.00	12.50
13	Hotdeugou	Olaf	2000-11-21	3	Représentant	33	27000.00	10.00
14	Odlavieille	Pacôme	2000-12-11	3	Représentant	34	25500.00	15.00
15	Amartakaldire	Quentin	2000-12-31	3	Représentant	35	23000.00	17.50
16	Traibien	Samira	2001-01-10	6	Secrétaire	41	15000.00	NULL

Exemple de récursivité

- Exemple : cherche les supérieurs de l'employé N°8 (parcours ascendant de l'arbre)

```
WITH RECURSIVE hierarchie( noemp, nom, nosupr) AS
(
    SELECT noemp, nom, nosupr
    FROM employes WHERE noemp= 8
    UNION ALL
    SELECT e.noemp, e.nom, e.nosupr
    FROM hierarchie h join employes e
        ON h.nosupr = e.noemp
)
SELECT * FROM hierarchie
ORDER BY noemp;
```

noemp	nom	prenom	embauche	nosupr	titre	nodept	salaire	tx_commission
1	Patamob	Adhémar	2000-03-26	NULL	President	50	50000.00	NULL
2	Zeublouze	Agathe	2000-04-16	1	Dir. Distrib	41	35000.00	NULL
3	Kuzbidon	Alex	2000-05-05	1	Dir. Vente	31	34000.00	NULL
4	Locale	Anasthasie	2000-05-25	1	Dir. Finance	10	36000.00	NULL
5	Teutmaronne	Armand	2000-06-14	1	Dir. Administr	50	36000.00	NULL
6	Zoudanikou	Debbie	2000-07-04	2	Chef Entrepôt	41	25000.00	NULL
7	Rivenbusse	Elsa	2000-07-24	2	Chef Entrepôt	42	24000.00	NULL
8	Ardelpic	Helmut	2000-08-13	2	Chef Entrepôt	43	23000.00	NULL
9	Peursconla	Humphrey	2000-09-02	2	Chef Entrepôt	44	22000.00	NULL
10	Vrante	Hélène	2000-09-22	2	Chef Entrepôt	45	21000.00	NULL
11	Enfaillite	Mélusine	2000-10-12	3	Représentant	31	25000.00	10.00
12	Eurktumeme	Odile	2000-11-01	3	Représentant	32	26000.00	12.50
13	Hotdeugou	Olaf	2000-11-21	3	Représentant	33	27000.00	10.00
14	Odlavieille	Pacôme	2000-12-11	3	Représentant	34	25500.00	15.00
15	Amertakeldire	Quentin	2000-12-31	3	Représentant	35	23000.00	17.50
16	Traibien	Samira	2001-01-10	6	Secrétaire	41	15000.00	NULL

La CTE **hierarchie** récupère le résultat de l'union à chaque itération

- | noemp | nom | nosupr |
|-------|----------|--------|
| 8 | Ardelpic | 2 |

- ON** **h.nosupr** = **e.noemp**

- | | | |
|---|-----------|---|
| 2 | Zeublouze | 1 |
| 8 | Ardelpic | 2 |

- | noemp | nom | nosupr |
|-------|-----------|--------|
| 1 | Patamob | NULL |
| 2 | Zeublouze | 1 |
| 8 | Ardelpic | 2 |

noemp	nom	prenom	embauche	nosupr	titre	nodept	salaire	tx_commission
1	Patamob	Adhémar	2000-03-28	NULL	President	50	50000.00	NULL
2	Zeublouze	Agathe	2000-04-15	1	Dir. Distrib	41	35000.00	NULL
3	Kuzbidon	Alex	2000-05-05	1	Dir. Vente	31	34000.00	NULL
4	Locale	Anasthasie	2000-05-25	1	Dir. Finance	10	36000.00	NULL
5	Teutmaronne	Armand	2000-06-14	1	Dir. Administr	50	36000.00	NULL
6	Soudanikou	Debbie	2000-07-04	2	Chef Entrepôt	41	25000.00	NULL
7	Rivenbusse	Elsa	2000-07-24	2	Chef Entrepôt	42	24000.00	NULL
8	Ardelpic	Helmut	2000-08-13	2	Chef Entrepôt	43	23000.00	NULL
9	Peursconla	Humphrey	2000-09-02	2	Chef Entrepôt	44	22000.00	NULL
10	Vrante	Hélène	2000-09-22	2	Chef Entrepôt	45	21000.00	NULL
11	Enfaillite	Mélusine	2000-10-12	3	Représentant	31	25000.00	10.00
12	Eurktumeme	Odile	2000-11-01	3	Représentant	32	26000.00	12.50
13	Hotdeugou	Olaf	2000-11-21	3	Représentant	33	27000.00	10.00
14	Odlavieille	Pacôme	2000-12-11	3	Représentant	34	25500.00	15.00
15	Amartakaldire	Quentin	2000-12-31	3	Représentant	35	23000.00	17.50
16	Traibien	Samira	2001-01-10	6	Secrétaire	41	15000.00	NULL

Exemple de récursivité

- Autre Exemple : on cherche cette fois les employés dont le supérieur est l'employé N° 3 (parcours descendant de l'arbre)

```
WITH RECURSIVE hierarchie(noemp, nom, nosupr) AS
```

```
(
```

```
    SELECT noemp, nom, nosupr
```

```
    FROM employes WHERE noemp= 3
```

```
    UNION ALL
```

```
    SELECT e.noemp, e.nom, e.nosupr
```

```
    FROM hierarchie h join employes e ON e.nosupr = h.noemp
```

```
)
```

```
SELECT * FROM hierarchie WHERE noemp != 3;
```

noemp	nom	nosupr
11	Enfaillite	3
12	Eurktumeme	3
13	Hotdeugou	3
14	Odlavieille	3
15	Amartakaldire	3

Exemple de récursivité

- On peut bien entendu utiliser les requêtes récursives dans des jointures, par exemple :

```
WITH RECURSIVE hierarchie(noemp, nom, nosupr, nodept) AS
(
    SELECT noemp, nom, nosupr, nodept
    FROM employes WHERE noemp= 8
    UNION ALL
    SELECT e.noemp, e.nom, e.nosupr, e.nodept
    FROM hierarchie h join employes e ON h.nosupr = e.noemp
)
SELECT * FROM hierarchie join departement using(nodept)
ORDER BY noemp;
```

nodept	noemp	nom	nosupr	nom	noregion
50	1	Patamob	NULL	Administration	1
41	2	Zeublouze	1	Distribution	1
43	8	Ardelpic	2	Distribution	3

Exemple de récursivité

- Ou avec un opérateur ensembliste;
 - Tous les employés qui ne sont pas sous les ordres de l'employé N°3

```
SELECT noemp, nom  
FROM employes
```

```
EXCEPT
```

```
( WITH RECURSIVE hierarchie(noemp, nom, nosupr) AS  
  ( SELECT noemp, nom, nosupr  
    FROM employes WHERE noemp= 3  
    UNION ALL  
    SELECT e.noemp, e.nom, e.nosupr  
    FROM hierarchie h join employes e ON e.nosupr = h.noemp  
  ) SELECT noemp, nom FROM hierarchie WHERE noemp != 3 )  
ORDER BY noemp;
```

noemp	nom
1	Patamob
2	Zeublouze
3	Kuzbidon
4	Locale
5	Teutmaronne
6	Zoudanlkou
7	Rivenbusse
8	Ardelpic
9	Peursconla
10	Vrante
16	Traibien
17	Fonfec
18	Fairant
19	Blaireur
20	Ajerre
21	Chmonfisse
22	Phototetedemort
23	Kaécouté
24	Adrouille-Toultan
25	Anchier

Exemple de récursivité : utilisation de 2 CTE récursives

WITH RECURSIVE

hierarchie(noemp, nom, nosupr) AS

```
(
  SELECT noemp, nom, nosupr
  FROM employes WHERE noemp= 2
```

UNION ALL

```
  SELECT e.noemp, e.nom, e.nosupr
  FROM hierarchie h join employes e ON e.nosupr = h.noemp
```

),

hierarchie2 (noemp, nom, nosupr) AS

```
(
  SELECT noemp, nom, nosupr
  FROM employes WHERE noemp= 10
```

UNION ALL

```
  SELECT e.noemp, e.nom, e.nosupr
  FROM hierarchie2 h join employes e ON e.nosupr = h.noemp
```

)

select * from *hierarchie*

except

select * from *hierarchie2*

noemp	nom	nosupr
2	Zeublouze	1
6	Zoudankou	2
7	Rivenbusse	2
8	Ardelpic	2
9	Peursconla	2
16	Traibien	6
17	Fonfec	6
18	Fairant	7
19	Blaireur	7
20	Ajerre	8
21	Chmonfisse	8
22	Phototetedemort	9
23	Kaécouté	9

A vertical decorative element on the left side of the slide, composed of a cluster of small blue, black, and yellow hexagons.

OLAP (*online analytical processing*)

Traitement analytique en ligne

A decorative element on the right side of the slide, consisting of a cluster of blue, black, and yellow hexagons.

A vertical decorative graphic on the left side of the slide, consisting of a series of colored dots (blue, yellow, black) and hexagons (blue, yellow) arranged in a pattern that tapers towards the top.

Avertissement

- La suite du support de ce cours est largement inspirer du support de cours (*devsqlpg.handout.pdf*)
- distribué par DALIBO (<https://dalibo.com>)
- sous LICENCE CREATIVE COMMONS BY-NC-SA 2.0 FR (<https://creativecommons.org/licenses/by-nc-sa/2.0/fr/legalcode>)

Agrégation étendue (OLAP)

- **OLAP (*online analytical processing*) :**
traitement analytique en ligne (couramment utilisé en informatique décisionnelle) permet l'analyse sur-le-champ d'informations selon plusieurs axes, dans le but d'obtenir des rapports de synthèse
- **Le GROUP BY ne construit qu'une seule partition des résultats**
Possibilité d'y adjoindre des opérateurs afin de visualiser plusieurs partitions en même temps

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage

Elles permettent de réaliser des calculs statistiques sur des groupements (comme le group by) mais le résultat n'entraîne pas le regroupement des enregistrements traités en un seul. Chaque enregistrement garde son identité propre

■ Utilisation :

- utiliser plusieurs critères d'agrégation dans la même requête
- utiliser des fonctions de classement
- faire référence à d'autres lignes de l'ensemble de données

PostgreSQL supporte les fonctions de fenêtrage depuis la version 8.4.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage

Nous utiliserons la table suivante pour les exemples

matricule	nom	service	salaire	type_emp
00000001	Dupuis	Direction	10000.00	cadre
00000040	Lebrac	Publication	3000.00	cadre
00000004	Fantasio	Courrier	4500.00	technicien
00000020	Lagaffe	Courrier	3000.00	technicien
00000006	Prunelle	Publication	4000.00	technicien



Agrégation étendue (OLAP)

- **Les fonctions de fenêtrage**
 - **Regroupement**
 - clause **OVER** (**PARTITION BY** <colonnes>)
 - **Utilité**
 - plusieurs critères de regroupement différents
 - avec des fonctions de calcul d'agrégats

La clause OVER permet de définir la façon dont les données sont regroupées - uniquement pour la colonne définie - avec la clause PARTITION BY.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Regroupement

■ Exemple

- On veut connaître la somme des salaires par service tout en affichant le matricule, le salaire et le service de l'employé !

```
SELECT matricule, salaire, service,  
SUM(salaire) OVER (PARTITION BY service)  
AS total_salaire_service  
FROM employes;
```

matricule	salaire	service	total_salaire_service
00000004	4500.00	Courrier	7500.00
00000020	3000.00	Courrier	7500.00
00000001	10000.00	Direction	10000.00
00000006	4000.00	Publication	7000.00
00000040	3000.00	Publication	7000.00

- Le terme PARTITION BY permet d'indiquer les critères de regroupement de la fenêtre sur laquelle on souhaite travailler
- Les calculs réalisés par cette requête sont identiques à ceux réalisés avec une agrégation utilisant GROUP BY. La principale différence est que l'on évite ici de perdre le détail des données tout en disposant des données agrégées dans le résultat de la requête.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Regroupement

■ Exemple

- Il est possible de réaliser plusieurs calculs d'agrégats sur des critères de regroupement différents !

```
SELECT matricule, salaire, service, type_emp,  
SUM(salaire) OVER (PARTITION BY service)  
                    AS total_salaire_service,  
AVG(salaire) OVER (PARTITION BY type_emp):: numeric(6,2)  
                    AS moyenne_salaire_type  
FROM employes;
```

matricule	salaire	service	type_emp	total_salaire_service	moyenne_salaire_type
00000001	10000.00	Direction	cadre	10000.00	6500.00
00000040	3000.00	Publication	cadre	7000.00	6500.00
00000004	4500.00	Courrier	technicien	7500.00	3833.33
00000020	3000.00	Courrier	technicien	7500.00	3833.33
00000006	4000.00	Publication	technicien	7000.00	3833.33

Agrégation étendue (OLAP)

- **Les fonctions de fenêtrage**

- **Tri**

- **OVER (ORDER BY ...)**

- **Utilité :**

- **numéroter les lignes :** `row_number()`
 - **classer des résultats :** `rank()`, `dense_rank()`
 - **faire appel à d'autres lignes du résultat :** `lead()`, `lag()`

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

- Exemple : numéroter les lignes en fonction du numéro de matricule, du plus petit au plus grand

```
SELECT row_number() OVER (ORDER BY matricule),  
matricule, nom  
FROM employes;
```

row_number	matricule	nom
1	00000001	Dupuis
2	00000004	Fantasio
3	00000006	Prunelle
4	00000020	Lagaffe
5	00000040	Lebrac

- La fonction `row_number()` permet de numéroter les lignes selon un critère de tri défini dans la clause OVER. L'ordre de tri de la clause OVER n'influence pas l'ordre de tri explicite d'une requête

```
SELECT row_number() OVER (ORDER BY matricule),  
matricule, nom  
FROM employes  
ORDER BY nom;
```

row_number	matricule	nom
1	00000001	Dupuis
2	00000004	Fantasio
4	00000020	Lagaffe
5	00000040	Lebrac
3	00000006	Prunelle

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

■ Exemple :

On dispose aussi de fonctions de classement, pour déterminer par exemple les employés les moins bien payés, avec les fonctions *rank()* et *dense_rank()*

```
SELECT matricule, nom, salaire, service,  
rank() OVER (ORDER BY salaire),  
dense_rank() OVER (ORDER BY salaire)  
FROM employes ;
```

matricule	nom	salaire	service	rank	dense_rank
00000040	Lebrac	3000.00	Publication	1	1
00000020	Lagaffe	3000.00	Courrier	1	1
00000006	Prunelle	4000.00	Publication	3	2
00000004	Fantasio	4500.00	Courrier	4	3
00000001	Dupuis	10000.00	Direction	5	4

La fonction de fenêtrage *rank()* renvoie le classement en autorisant des trous dans la numérotation, et *dense_rank()* le classement sans trous.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

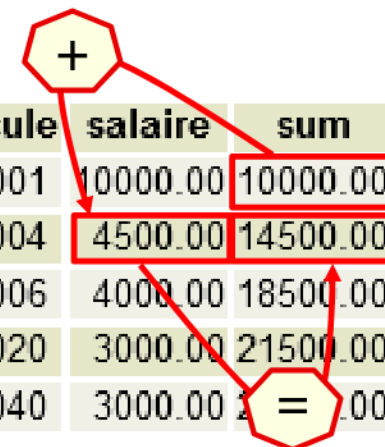
■ Tri avec une somme (SUM) :

```
SELECT matricule, salaire,  
SUM(salaire) OVER (ORDER BY matricule)  
FROM employes;
```

Lorsque l'on utilise une clause de tri, la portion de données visible par l'opérateur d'agrégat correspond aux données comprises entre la première ligne examinée et la ligne courante, la somme se fait de manière séquentiel et progressive

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00



Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

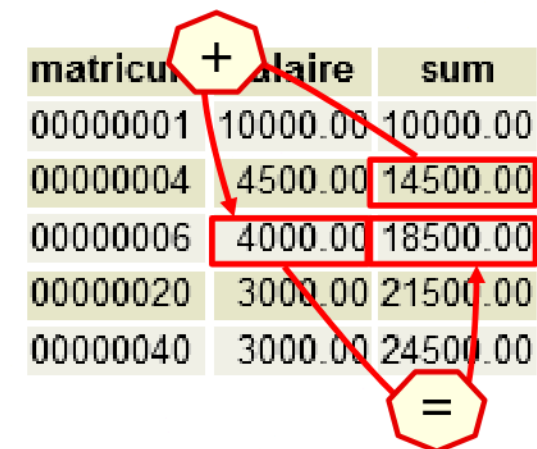
■ Tri avec une somme (SUM) :

```
SELECT matricule, salaire,  
SUM(salaire) OVER (ORDER BY matricule)  
FROM employes;
```

Lorsque l'on utilise une clause de tri, la portion de données visible par l'opérateur d'agrégat correspond aux données comprises entre la première ligne examinée et la ligne courante, la somme se fait de manière séquentiel et progressive

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00

matricule	+	salaire	=	sum
00000001		10000.00		10000.00
00000004		4500.00		14500.00
00000006		4000.00		18500.00
00000020		3000.00		21500.00
00000040		3000.00		24500.00



Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

■ Tri avec une somme (SUM) :

```
SELECT matricule, salaire,  
SUM(salaire) OVER (ORDER BY matricule)  
FROM employes;
```

Lorsque l'on utilise une clause de tri, la portion de données visible par l'opérateur d'agrégat correspond aux données comprises entre la première ligne examinée et la ligne courante, la somme se fait de manière séquentiel et progressive

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00

The diagram illustrates the cumulative sum calculation. A red hexagon with a '+' sign is positioned over the 'salaire' column of the first row (00000001). Red arrows point from this '+' sign to the 'salaire' and 'sum' columns of the second row (00000004). Another red arrow points from the 'sum' column of the second row to the 'sum' column of the third row (00000006). A red hexagon with an '=' sign is positioned below the 'sum' column of the third row, with red arrows pointing from the 'salaire' and 'sum' columns of the third row to it. The 'sum' column of the third row is highlighted with a red box.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI

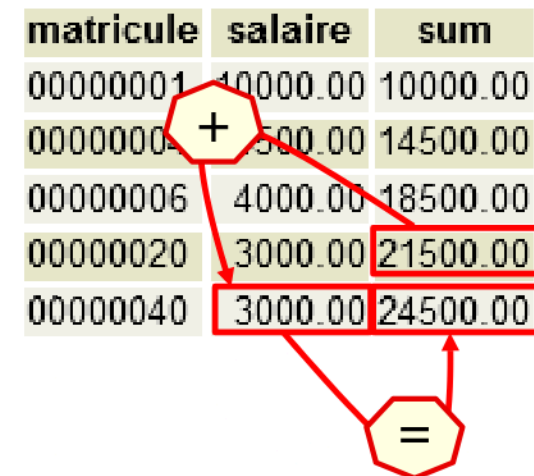
■ Tri avec une somme (SUM) :

```
SELECT matricule, salaire,  
SUM(salaire) OVER (ORDER BY matricule)  
FROM employes;
```

Lorsque l'on utilise une clause de tri, la portion de données visible par l'opérateur d'agrégat correspond aux données comprises entre la première ligne examinée et la ligne courante, la somme se fait de manière séquentiel et progressive

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00

matricule	salaire	sum
00000001	10000.00	10000.00
00000004	4500.00	14500.00
00000006	4000.00	18500.00
00000020	3000.00	21500.00
00000040	3000.00	24500.00



Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI et regroupement

■ On peut combiner les deux

```
SELECT ...  
    <agregation> OVER (PARTITION BY <colonnes>  
                        ORDER BY <colonnes>)  
FROM <liste_tables>  
WHERE <predicats>
```

*La norme impose de placer la clause
PARTITION BY avant la clause ORDER
BY*

■ Utilité :

■ travailler sur des jeux de données ordonnés et isolés les uns des autres :

Il est possible de combiner les clauses de fenêtrage PARTITION BY et ORDER BY. Cela permet d'isoler des jeux de données entre eux avec la clause PARTITION BY, tout en appliquant un critère de tri avec la clause ORDER BY sur le groupement.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : TRI et regroupement

■ Exemple :

```
SELECT continent, pays, population,  
rank() OVER (PARTITION BY continent  
ORDER BY population DESC)  
AS rang  
FROM population;
```

Le groupement se fait sur le continent et ensuite on applique un tri décroissant sur la population. La numérotation du rang() est faite sur chaque groupement

continent	pays	population	rang
Afrique	Nigéria	173.6	1
Afrique	Éthiopie	94.1	2
Afrique	Égypte	82.1	3
Afrique	Rép. dém. du Congo	67.5	4
(...)			
Amérique du Nord	États-Unis	320.1	1
Amérique du Nord	Canada	35.2	2
(...)			

Agrégation étendue (OLAP)

- **Les fonctions de fenêtrage : Fonctions analytiques**
 - PostgreSQL dispose d'un certain nombre de fonctions analytiques
 - Utilité :
 - faire référence à d'autres lignes du même ensemble
 - évite les auto-jointures complexes et lentes

Sans les fonctions analytiques, il était difficile en SQL d'écrire des requêtes nécessitant de faire appel à des données provenant d'autres lignes que la ligne courante

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **Exemple** : pour renvoyer la liste détaillée de tous les employés ET le salaire le plus élevé du service auquel il appartient, on peut utiliser la fonction *first_value()* :

```
SELECT matricule, nom, salaire, service,  
first_value(salaire) OVER (PARTITION BY service ORDER BY salaire DESC)  
AS salaire_maximum_service  
FROM employes ;
```

matricule	nom	salaire	service	salaire_maximum_service
00000004	Fantasio	4500.00	Courrier	4500.00
00000020	Lagaffe	3000.00	Courrier	4500.00
00000001	Dupuis	10000.00	Direction	10000.00
00000006	Prunelle	4000.00	Publication	4000.00
00000040	Lebrac	3000.00	Publication	4000.00

Groupement sur le service et tri décroissant sur le salaire. Ensuite avec la fonction first_value(salaire) on affiche la première valeur du groupement, cad le salaire la plus élevé du groupement

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

■ Il existe également les fonctions suivantes :

- *last_value(colonne)* : renvoie la dernière valeur de la colonne du groupement ;
- *first_value(colonne)* : renvoie la première valeur de la colonne du groupement ;
- *nth(colonne, n)* : retourne la valeur placée à n-ème position (en partant de 1);
- *lag(colonne, n)* : renvoie la valeur située en n-ème position de la colonne du groupement **avant** la ligne en cours pour la colonne ;
- *lead(colonne, n)* : renvoie la valeur située en n-ème position de la colonne du groupement **après** la ligne en cours pour la colonne ;
 - *pour ces deux fonctions, le n est facultatif et vaut 1 par défaut ;*
 - *ces deux fonctions acceptent un 3ème argument facultatif spécifiant la valeur à renvoyer si aucune valeur n'est trouvée en n-ème position avant ou après. Par défaut, NULL sera renvoyé.*

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **LAG(...)** : renvoie la valeur située en n-ème position **avant** la ligne en cours

- **Exemple :**

```
SELECT matricule, nom, salaire, service,  
lag(salaire) OVER (PARTITION BY service ORDER BY salaire DESC) AS salaire_maximum_service  
FROM employes ;
```

lag(salaire) renvoie la valeur de la ligne précédente,
salaire est un numeric, donc *lag(salaire)* renvoie
un numeric.

lag(salaire) est équivalent à *lag(salaire,1)*

lag(salaire,2) renvoie la valeur du salaire 2 lignes
avant

matricule	nom	salaire	service	salaire_maximum_service
00000034	Fan	4700.00	Courrier	NULL
00000004	Fantasio	4500.00	Courrier	4700.00
00000025	Bidule	4000.00	Courrier	4500.00
00000020	Lagaffe	3000.00	Courrier	4000.00
00000021	Blanc	15000.00	Direction	NULL
00000011	Dupond	12000.00	Direction	15000.00
00000001	Dupuis	10000.00	Direction	12000.00
00000036	Pruneau	5500.00	Publication	NULL
00000006	Prunelle	4000.00	Publication	5500.00
00000043	Lebon	3400.00	Publication	4000.00
00000040	Lebrac	3000.00	Publication	3400.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **LAG(...)** : renvoie la valeur située en n-ème position **avant** la ligne en cours

- **Exemple :**

```
SELECT matricule, nom, salaire, service,  
lag(salaire,1,0::numeric) OVER (PARTITION BY service ORDER BY salaire DESC) AS salaire_maximum_service  
FROM employes ;
```

Il est possible de renvoyer une valeur plutôt que NULL, si pas de ligne avant la ligne courante.

lag(salaire,1,0::numeric) renvoie un numeric, le transtypage est obligatoire, la valeur de retour doit être du même type que la colonne référencée par lag(...) ici salaire

matricule	nom	salaire	service	salaire_maximum_service
00000034	Fan	4700.00	Courrier	0
00000004	Fantasio	4500.00	Courrier	4700.00
00000025	Bidule	4000.00	Courrier	4500.00
00000020	Lagaffe	3000.00	Courrier	4000.00
00000021	Blanc	15000.00	Direction	0
00000011	Dupond	12000.00	Direction	15000.00
00000001	Dupuis	10000.00	Direction	12000.00
00000036	Pruneau	5500.00	Publication	0
00000006	Prunelle	4000.00	Publication	5500.00
00000043	Lebon	3400.00	Publication	4000.00
00000040	Lebrac	3000.00	Publication	3400.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **LAG(...)** : renvoie la valeur située en n-ème position **avant** la ligne en cours

- **Exemple :**

```
SELECT matricule, nom, salaire, service,  
lag(salaire,2) OVER (PARTITION BY service ORDER BY salaire DESC) as salaire_2  
FROM employes ;
```

lag(salaire,2) renvoie la valeur du salaire 2 lignes avant

matricule	nom	salaire	service	salaire_2
00000034	Fan	4700.00	Courier	NULL
00000004	Fantasio	4500.00	Courier	NULL
00000025	Bidule	4000.00	Courier	4700.00
00000020	Lagaffe	3000.00	Courier	4500.00
00000021	Blanc	15000.00	Direction	NULL
00000011	Dupond	12000.00	Direction	NULL
00000001	Dupuis	10000.00	Direction	15000.00
00000036	Pruneau	5500.00	Publication	NULL
00000006	Prunelle	4000.00	Publication	NULL
00000043	Lebon	3400.00	Publication	5500.00
00000040	Lebrac	3000.00	Publication	4000.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **LEAD(...)** : renvoie la valeur située en n-ème position **après** la ligne en cours

- **Exemple :**

```
SELECT matricule, nom, salaire, service,  
lead(salaire) OVER (PARTITION BY service ORDER BY salaire DESC) as salaire_juste_inf  
FROM employes ;
```

matricule	nom	salaire	service	salaire_juste_inf
00000034	Fan	4700.00	Courrier	4500.00
00000004	Fantasio	4500.00	Courrier	4000.00
00000025	Bidule	4000.00	Courrier	3000.00
00000020	Lagaffe	3000.00	Courrier	NULL
00000021	Blanc	15000.00	Direction	12000.00
00000011	Dupond	12000.00	Direction	10000.00
00000001	Dupuis	10000.00	Direction	NULL
00000036	Pruneau	5500.00	Publication	4000.00
00000006	Prunelle	4000.00	Publication	3400.00
00000043	Lebon	3400.00	Publication	3000.00
00000040	Lebrac	3000.00	Publication	NULL

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- `nth_value(colonne, n)` : retourne la valeur placée à nième position (en partant de 1)

- Exemple :

```
SELECT matricule, nom, salaire, service,  
nth_value(salaire,1) OVER (PARTITION BY service ORDER BY salaire DESC) as salaire_max  
FROM employes ;
```

*Cela revient à retourner le salaire maximum
du groupement.*

*Si on veut le salaire minimum il suffit de faire
un tri croissant sur le salaire :*

`OVER (PARTITION BY service ORDER BY salaire ASC)`

matricule	nom	salaire	service	salaire max
00000034	Fan	4700.00	Courrier	4700.00
00000004	Fantasio	4500.00	Courrier	4700.00
00000025	Bidule	4000.00	Courrier	4700.00
00000020	Lagaffe	3000.00	Courrier	4700.00
00000021	Blanc	15000.00	Direction	15000.00
00000011	Dupond	12000.00	Direction	15000.00
00000001	Dupuis	10000.00	Direction	15000.00
00000036	Pruneau	5500.00	Publication	5500.00
00000006	Prunelle	4000.00	Publication	5500.00
00000043	Lebon	3400.00	Publication	5500.00
00000040	Lebrac	3000.00	Publication	5500.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- `nth_value(colonne, n)` : retourne la n-ème valeur (en comptant à partir de 1)
 - Autre exemple : on veut retourner le salaire juste en dessous celui le plus élevé

```
select matricule, nom, salaire, service,
nth_value(salaire, 2) over(partition by service order by salaire desc) as salaire_2
from employes
```

Cela revient à retourner le salaire qui est juste en dessous le salaire le plus élevé du groupement.

Le problème c'est que l'opération se réalise

À partir de la 2^{ème} valeur du groupement !!

matricule	nom	service	salaire	type_emp
00000001	Dupuis	Direction	10000.00	cadre
00000040	Lebrac	Publication	3000.00	cadre
00000004	Fantasio	Courier	4500.00	technicien
00000020	Lagaffe	Courier	3000.00	technicien
00000006	Prunelle	Publication	4000.00	technicien

matricule	nom	salaire	service	salaire_2
00000034	Fan	4700.00	Courier	NULL
00000004	Fantasio	4500.00	Courier	4500.00
00000025	Bidule	4000.00	Courier	4500.00
00000020	Lagaffe	3000.00	Courier	4500.00
00000021	Blanc	15000.00	Direction	NULL
00000011	Dupond	12000.00	Direction	12000.00
00000001	Dupuis	10000.00	Direction	12000.00
00000036	Pruneau	5500.00	Publication	NULL
00000006	Prunelle	4000.00	Publication	4000.00
00000043	Lebon	3400.00	Publication	4000.00
00000040	Lebrac	3000.00	Publication	4000.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- **nth_value(colonne, n)** : retourne la n-ème valeur (en comptant à partir de 1)

En ajouter la clause **RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**

On va pouvoir réaliser l'opération sur toutes les valeurs du groupement

- **RANGE** : Cela indique que la fenêtre sera basée sur les valeurs de la colonne spécifiée dans la clause **ORDER BY** de la fonction analytique, et non sur les numéros de ligne.
- **UNBOUNDED PRECEDING** : Cela signifie que la fenêtre commence au début de la partition. En d'autres termes, la fenêtre inclut toutes les lignes avant (ou jusqu'à) la ligne courante dans la partition. Si la partition est définie, elle commence depuis la première ligne de cette partition.
- **UNBOUNDED FOLLOWING** : Cela signifie que la fenêtre s'étend jusqu'à la fin de la partition. Cela inclut toutes les lignes après (ou jusqu'à) la ligne courante, jusqu'à la dernière ligne de la partition.

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Fonctions analytiques

- `nth_value(colonne, n)` : retourne la n-ème valeur (en comptant à partir de 1)

En ajouter la clause **RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**

```
select matricule, nom, salaire, service,  
nth_value(salaire, 2) over(partition by service order by salaire desc  
RANGE between UNBOUNDED PRECEDING AND  
UNBOUNDED FOLLOWING) as salaire_2  
from employes
```

matricule	nom	service	salaire	type_emp
00000001	Dupuis	Direction	10000.00	cadre
00000040	Lebrac	Publication	3000.00	cadre
00000004	Fantasio	Courrier	4500.00	technicien
00000020	Lagaffe	Courrier	3000.00	technicien
00000006	Prunelle	Publication	4000.00	technicien

matricule	nom	salaire	service	salaire 2
00000034	Fan	4700.00	Courrier	4500.00
00000004	Fantasio	4500.00	Courrier	4500.00
00000025	Bidule	4000.00	Courrier	4500.00
00000020	Lagaffe	3000.00	Courrier	4500.00
00000021	Blanc	15000.00	Direction	12000.00
00000011	Dupond	12000.00	Direction	12000.00
00000001	Dupuis	10000.00	Direction	12000.00
00000036	Pruneau	5500.00	Publication	4000.00
00000006	Prunelle	4000.00	Publication	4000.00
00000043	Lebon	3400.00	Publication	4000.00
00000040	Lebrac	3000.00	Publication	4000.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Clause WINDOWS

Il arrive que l'on ait besoin d'utiliser plusieurs fonctions de fenêtrage au sein d'une même requête qui utilisent la même définition de fenêtre (même clause PARTITION BY et/ou ORDER BY). Afin d'éviter de dupliquer cette clause, il est possible de définir une fenêtre nommée et de l'utiliser à plusieurs endroits de la requête.

■ Exemple 1 :

```
SELECT matricule, salaire, service, type_emp,  
       sum(salaire) OVER w1 AS "somme salaire",  
       avg(salaire) OVER w1 ::numeric(8,2) AS "moyenne salaire",  
       max(salaire) OVER w1 AS "max salaire"  
FROM employes  
WINDOW w1 AS (PARTITION BY service ORDER BY service)
```

matricule	salaire	service	type_emp	somme salaire	moyenne salaire	max salaire
00000034	4700.00	Courrier	technicien	16200.00	4050.00	4700.00
00000025	4000.00	Courrier	technicien	16200.00	4050.00	4700.00
00000004	4500.00	Courrier	technicien	16200.00	4050.00	4700.00
00000020	3000.00	Courrier	technicien	16200.00	4050.00	4700.00
00000001	10000.00	Direction	cadre	37000.00	12333.33	15000.00
00000011	12000.00	Direction	cadre	37000.00	12333.33	15000.00
00000021	15000.00	Direction	cadre	37000.00	12333.33	15000.00
00000043	3400.00	Publication	cadre	15900.00	3975.00	5500.00
00000006	4000.00	Publication	technicien	15900.00	3975.00	5500.00
00000040	3000.00	Publication	cadre	15900.00	3975.00	5500.00
00000036	5500.00	Publication	technicien	15900.00	3975.00	5500.00

Agrégation étendue (OLAP)

■ Les fonctions de fenêtrage : Clause WINDOWS

Il est possible de définir de multiples définitions de fenêtres au sein d'une même requête, et qu'une définition de fenêtre peut surcharger la clause ORDER BY si la définition parente ne l'a pas définie

■ Exemple 2 :

```
SELECT matricule, nom, salaire, service,  
rank() OVER w_asc ,  
dense_rank() OVER w_desc  
FROM employes  
WINDOW w AS (PARTITION BY service),  
        w_asc AS (w ORDER BY salaire),  
        w_desc AS (w ORDER BY salaire DESC);
```

matricule	nom	salaire	service	rank	dense rank
00000034	Fan	4700.00	Courrier	4	1
00000004	Fantasio	4500.00	Courrier	3	2
00000025	Bidule	4000.00	Courrier	2	3
00000020	Lagaffe	3000.00	Courrier	1	4
00000021	Blanc	15000.00	Direction	3	1
00000011	Dupond	12000.00	Direction	2	2
00000001	Dupuis	10000.00	Direction	1	3
00000036	Pruneau	5500.00	Publication	4	1
00000006	Prunelle	4000.00	Publication	3	2
00000043	Lebon	3400.00	Publication	2	3
00000040	Lebrac	3000.00	Publication	1	4

La sélection de partitions avec *GROUPING SETS*

- Il est possible de définir dans une même requête un calcul sur plusieurs groupements différents, en listant les colonnes sur lesquelles on désire réaliser les groupements successifs ;

- Utilisation de la syntaxe :

SELECT

FROM

GROUP BY GROUPING SETS(*liste des groupements à réaliser*)

La sélection de partitions avec *GROUPING SETS*

- *Pour les exemples qui suivent nous nous appuierons sur la table suivante, elle contient le nombre de modification (updates), suppression (deletes), insertion (inserts) et de transaction (transactions) réalisés par un utilisateur (appnumber) et pour un jour donné (day)*

appnumber	day	inserts	updates	deletes	transactions
1	2017-09-25	1	8	15	22
1	2017-09-26	29	36	43	50
1	2017-09-27	57	64	71	78
2	2017-09-25	85	92	99	106
2	2017-09-26	113	120	127	134
2	2017-09-27	141	148	155	162
3	2017-09-25	169	176	183	190
3	2017-09-26	197	204	211	218
3	2017-09-27	225	232	239	246

La sélection de partitions avec *GROUPING SETS*

- Si nous voulons connaître le nombre d'insertion, de modification, de suppression et de transaction réalisé par jour et ensuite par utilisateur, avec un group by nous devons réaliser deux requêtes :

Requête 1 : pour un groupement par *utilisateur*

```
SELECT appnumber, SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY appnumber;
```

appnumber	nb insert	nb update	nb delete	nb transaction
1	87	108	129	150
3	591	612	633	654
2	339	360	381	402

La sélection de partitions avec *GROUPING SETS*

Requête 2 : pour un groupement par jour

```
SELECT appnumber as "par jour",  
       SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY day;
```

par jour	nb insert	nb update	nb delete	nb transaction
2017-09-25	255	276	297	318
2017-09-26	339	360	381	402
2017-09-27	423	444	465	486

La sélection de partitions avec *GROUPING SETS*

- Il est possible avec un *GROUPING SETS* de réaliser ces deux requêtes en une seule :

```
SELECT appnumber, day, SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY GROUPING SETS( appnumber, day);
```

	appnumber	day	nb insert	nb update	nb delete	nb transaction
Groupement par utilisateur	1	NULL	87	108	129	150
	2	NULL	339	360	381	402
	3	NULL	591	612	633	654
Groupement par jour	NULL	2017-09-25	255	276	297	318
	NULL	2017-09-26	339	360	381	402
	NULL	2017-09-27	423	444	465	486

La sélection de partitions avec *GROUPING SETS*

- On peut ajouter la somme globale des update, insert, update et transaction

```
SELECT appnumber, day, SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY GROUPING SETS (appnumber, day, ());
```

Groupement par utilisateur

Pas de groupement (nombre total de chaque colonne)

Groupement par jour

appnumber	day	nb insert	nb update	nb delete	nb transaction
1	NULL	87	108	129	150
2	NULL	339	360	381	402
3	NULL	591	612	633	654
NULL	NULL	1017	1080	1143	1206
NULL	2017-09-25	255	276	297	318
NULL	2017-09-26	339	360	381	402
NULL	2017-09-27	423	444	465	486

Le chemin des partitions avec ROLLUP

- Il est possible de définir un chemin de partition, cad une combinaison de partition avec ROLLUP :

ROLLUP (e1, e2, e3, ...)

Cela revient à faire
GROUPING SETS

(
 (e1, e2, e3, ...), -- groupement sur toutes les colonnes
 ...
 (e1, e2), -- groupement sur les deux premières
 (e1), -- groupement sur la première
 () -- pas de groupement
)

Le chemin des partitions avec ROLLUP

■ Exemple

Si nous voulons connaître le nombre d'insertion, de modification, de suppression et de transaction réalisé par utilisateur et par jour et ensuite par utilisateur :

```
SELECT appnumber, day, SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY ROLLUP(appnumber, day);
```

Cela revient à faire

GROUPING SETS ((appnumber, day), (appnumber), ())

Le chemin des partitions avec ROLLUP

```
SELECT appnumber, day, SUM(INSERTS) as "nb insert",  
       SUM(UPDATES) as "nb update",  
       SUM(DELETES) as "nb delete",  
       SUM(TRANSACTIONS) as "nb transaction"  
FROM db_details  
GROUP BY ROLLUP(appnumber, day);
```

Groupement par appnumber et day

Groupement par appnumber

Pas de groupement (nombre total de chaque colonne)

appnumber	day	nb insert	nb update	nb delete	nb transaction
1	2017-09-25	1	8	15	22
2	2017-09-25	85	92	99	106
3	2017-09-25	169	176	183	190
1	2017-09-26	29	36	43	50
2	2017-09-26	113	120	127	134
3	2017-09-26	197	204	211	218
1	2017-09-27	57	64	71	78
2	2017-09-27	141	148	155	162
3	2017-09-27	225	232	239	246
1	NULL	87	108	129	150
2	NULL	339	360	381	402
3	NULL	591	612	633	654
NULL	NULL	1017	1080	1143	1206

Toutes les partitions avec CUBE

- Pour illustrer cette commande nous allons nous appuyer sur la table suivante :

d1	d2	d3	v
a	c	e	40
a	c	f	6
a	d	e	44
a	d	f	8
b	c	e	38
b	c	f	12
b	d	e	38
b	d	f	36

Toutes les partitions avec CUBE

- Avec la commande CUBE(..) nous allons pouvoir combiner plusieurs groupement :

Par exemple

```
... GROUP BY CUBE ( a, b, c )
```

Equivaux à

```
... GROUPING SETS
```

```
(  
    ( a, b, c ),  
    ( a, b      ),  
    ( a,      c ),  
    ( a          ),  
    (      b, c ),  
    (          b ),  
    (          c ),  
    (              )  
)
```

Toutes les partitions avec CUBE

- Revenons à notre table, par exemple

```
SELECT d1, d2, d3, sum(v)
FROM test
GROUP BY CUBE ( d1, d2, d3 );
```

d1	d2	d3	sum
a	c	e	40
a	c	f	6
a	c	NULL	46
a	d	e	44
a	d	f	8
a	d	NULL	52
a	NULL	NULL	98
b	c	e	38
b	c	f	12
b	c	NULL	50
b	d	e	38
b	d	f	36
b	d	NULL	74
b	NULL	NULL	124
NULL	NULL	NULL	222
a	NULL	e	84
b	NULL	e	76
NULL	NULL	e	160
a	NULL	f	14
b	NULL	f	48
NULL	NULL	f	62
NULL	c	e	78
NULL	c	f	18
NULL	c	NULL	96
NULL	d	e	82
NULL	d	f	44
NULL	d	NULL	126

A vertical decorative graphic on the left side of the slide, composed of a cluster of blue and yellow hexagons of various sizes, some solid and some outlined.

C'est fini ...

- 1.1. Donnez les feuilles de la hiérarchie (les nœuds qui ne sont pas racine) des thèmes (identifiant, libellé).

```
SELECT      IdT, Libelle
FROM        THEME
WHERE       IdT NOT IN (SELECT IdTPere
                        FROM THEME
                        WHERE IdTPere IS NOT NULL);
```

Ou avec une sous requête corrélée et un NOT EXISTS

```
SELECT      IdT, Libelle
FROM        THEME T
WHERE       NOT EXISTS (SELECT *
                        FROM THEME
                        WHERE IdTPere =T.idt)
```

- 1.2. Donnez toute la hiérarchie (au dessus de lui) (identifiant, libellé) de classification du thème Jointure externe (Triez cette liste du thème le plus général au thème le plus spécifique).

```
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'JOINTURE EXTERNE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON h.idtpere= t.idt
)
SELECT * FROM hierarchie order by 1;
```

- 1.3. Donnez les racines de la hiérarchie des thèmes (identifiant, libellé).(ceux qui n'ont pas de père)

```
SELECT IdT, Libelle
FROM    THEME
WHERE   IdTPere IS NULL;
```

- 1.4. Quels sont les thèmes (identifiant, libellé) spécialisant (i.e. plus précis), le thème Langage de requêtes ? On ne veut pas le thème général (cad le thème Langage de requêtes). Toute la hiérarchie et ensuite que les fils directs

```
Toute la hiérarchie
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
( SELECT IdT, Libelle,idtpere
  FROM THEME WHERE Libelle = 'LANGAGE DE REQUETES'
  UNION ALL
  SELECT t.IdT, t.Libelle,t.idtpere
  FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT IdT, Libelle FROM hierarchie
where Libelle != 'LANGAGE DE REQUETES'
order by 1;
```

ou uniquement les fils directs

```
WITH RECURSIVE hierarchie(IdT, Libelle, idtpere) AS
(
    SELECT IdT, Libelle, idtpere
    FROM THEME WHERE Libelle = 'LANGAGE DE REQUETES'
    UNION ALL
    SELECT t.IdT, t.Libelle, t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
    where t.idtpere=(select idt
                     FROM THEME
                     WHERE Libelle = 'LANGAGE DE REQUETES')
)
SELECT IdT, Libelle FROM hierarchie
where idt!= (select idt from theme where Libelle = 'LANGAGE DE REQUETES')
order by 1;
```

ou uniquement les fils directs SANS RECURSIVITE !!!!

```
SELECT IdT, Libelle FROM theme
where idtpere=(select idt from theme where Libelle = 'LANGAGE DE REQUETES')
order by 1;
```

- 1.5. Quels sont les thèmes (identifiant, libellé) rattachés directement ou pas au thème Jointure à l'exception du thème Jointure imbriquée et de ses sous-thèmes ?

```
WITH RECURSIVE hierarchie(IdT, Libelle, idtpere) AS
(
    SELECT IdT, Libelle, idtpere
    FROM THEME WHERE Libelle = 'JOINTURE'
    UNION ALL
    SELECT t.IdT, t.Libelle, t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT * FROM hierarchie
where idt not in(WITH RECURSIVE hierarchie(IdT, Libelle, idtpere) AS
(
    SELECT IdT, Libelle, idtpere
    FROM THEME WHERE Libelle = 'JOINTURE IMBRIQUEE'
    UNION ALL
    SELECT t.IdT, t.Libelle, t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT IdT FROM hierarchie)
order by 1;
```

ou avec un except

```
(
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'JOINTURE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT * FROM hierarchie )

EXCEPT

(
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'JOINTURE IMBRIQUEE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT * FROM hierarchie
)
```

Ou encore plus simple!

```
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'JOINTURE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
    WHERE t.Libelle != 'JOINTURE IMBRIQUEE'
)
SELECT * FROM hierarchie ;
```

- 1.6. Quels sont les thèmes (identifiant, libellé) rattachés directement ou pas au thème Jointure à l'exception du thème Jointure imbriquée ?

```
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'JOINTURE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT IdT, Libelle
FROM hierarchie
where Libelle != 'JOINTURE IMBRIQUEE' order by 1;
```

- 1.7. Donnez la liste des questions (identifiant, numéro de TP) se rapportant à un thème rattaché directement ou non au thème Jointure.

```
SELECT distinct QUESTION.IdQ, NumTP
FROM QUESTION join THEMQUEST using(IdQ )
WHERE IdT IN (
    WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
    (
        SELECT IdT, Libelle,idtpere
        FROM THEME WHERE Libelle = 'JOINTURE'
        UNION ALL
        SELECT t.IdT, t.Libelle,t.idtpere
        FROM hierarchie h join THEME t ON t.idtpere= h.idt
    )
    SELECT IdT
    FROM hierarchie
)
```

- 1.8. Quels sont les thèmes (identifiant, libellé) généralisant (ascendant) les thèmes Jointure **ou** Sélection simple ?
Donner deux formulations

1^{ère} formulation

```
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle IN('JOINTURE','SELECTION SIMPLE')
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON h.idtpere= t.idt
)
SELECT distinct IdT, Libelle,idtpere
FROM hierarchie order by 1;
```

2^{ème} formulation

```

WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME
    WHERE Libelle = 'JOINTURE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON h.idtpere= t.idt
)
SELECT distinct IdT, Libelle,idtpere
FROM hierarchie
)

UNION

(
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'SELECTION SIMPLE'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON h.idtpere= t.idt
)
SELECT distinct IdT, Libelle,idtpere
FROM hierarchie
)
order by 1

```

1.9. Quels sont les thèmes (identifiant, libellé) généralisant les thèmes Jointure et Sélection simple ?

```

(
    WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
    (
        SELECT IdT, Libelle,idtpere
        FROM THEME WHERE Libelle = 'SELECTION SIMPLE'
        UNION ALL
        SELECT t.IdT, t.Libelle,t.idtpere
        FROM hierarchie h join THEME t ON h.idtpere= t.idt
    )
    SELECT distinct IdT, Libelle
    FROM hierarchie
)
INTERSECT
(
    WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
    (
        SELECT IdT, Libelle,idtpere
        FROM THEME WHERE Libelle = 'JOINTURE'
        UNION ALL
        SELECT t.IdT, t.Libelle,t.idtpere
        FROM hierarchie h join THEME t ON h.idtpere= t.idt
    )
    SELECT distinct IdT, Libelle
    FROM hierarchie
)

```

1.10. Donnez les nom et prénom des étudiants ayant fait toutes les questions du TP 1. Donnez deux formulations SQL

Quels sont les étudiants ayant fait autant de questions du TP n° 1 qu'il en existe.

```
SELECT ETUDIANT.NumEt, Nom, Prenom
FROM ETUDIANT join EVALUATION using (NumEt)
              join QUESTION using (IdQ)
WHERE NumTP = 1
GROUP BY ETUDIANT.NumEt, Nom, Prenom
HAVING COUNT (*) =
        (SELECT COUNT (*)
         FROM QUESTION
         WHERE NumTP = 1)
```

Quels sont les étudiants tels qu'il n'existe aucune question du TP n° 1 qu'ils n'aient pas faite.

```
SELECT NumEt, Nom, Prenom
FROM ETUDIANT E
WHERE NOT EXISTS
      (SELECT *
       FROM QUESTION Q
       WHERE NumTP = 1 AND NOT EXISTS
            (SELECT *
             FROM EVALUATION
             WHERE E.NumEt = EVALUATION.NumEt
             AND EVALUATION.IdQ = Q.IdQ))
```

1.11. Pour chaque groupe, donner le nombre d'étudiants ayant un BAC S et le nombre d'étudiants ayant un BAC STI.

```
SELECT distinct Groupe,
      (SELECT COUNT(*) FROM ETUDIANT where groupe=E.groupe and typebac='S') as Etu_S ,
      (SELECT COUNT(*) FROM ETUDIANT where groupe=E.groupe and typebac='STI') as
      Etu_STI
from etudiant E
order by 1
```

1.12. Quels sont les groupes dans lesquels tous les types de BAC sont représentés ?

Quels sont les groupes ayant autant de Types de BAC qu'il en existe.

```
SELECT Groupe
FROM ETUDIANT
GROUP BY Groupe
HAVING COUNT(DISTINCT TypeBAC) =(SELECT COUNT(DISTINCT TypeBAC) FROM ETUDIANT)
```

1.13. Quels sont les groupes pour lesquels au moins un étudiant a été évalué pour chaque TP. Donner deux formulations

Quels sont les groupes pour lesquels il y a autant de TP évalués qu'il en existe.

```
SELECT Groupe
FROM ETUDIANT join EVALUATION using (NumEt )
              join QUESTION using (IdQ)
GROUP BY Groupe, numet
HAVING COUNT(DISTINCT NumTP) = (SELECT COUNT(DISTINCT NumTP)
                                FROM QUESTION);
```

Quels sont les groupes tels qu'il n'existe aucun TP pour lequel qu'ils n'aient pas été évalué.

```
SELECT DISTINCT Groupe
FROM   ETUDIANT E
WHERE  NOT EXISTS (SELECT *
                  FROM QUESTION Q
                  WHERE NOT EXISTS (SELECT *
                                   FROM ETUDIANT join EVALUATION using (NumEt )
                                   join QUESTION using (IdQ)
                                   WHERE E.NumEt = ETUDIANT.NumEt
                                   AND   QUESTION.NumTP = Q.NumTP))
```

- 1.14. Quels sont les thèmes (identifiant, libellé) rattachés directement ou pas au thème SQL LMD et qui ne sont utilisés par aucune question ?

```
WITH RECURSIVE hierarchie(IdT, Libelle,idtpere) AS
(
    SELECT IdT, Libelle,idtpere
    FROM THEME WHERE Libelle = 'SQL LMD'
    UNION ALL
    SELECT t.IdT, t.Libelle,t.idtpere
    FROM hierarchie h join THEME t ON t.idtpere= h.idt
)
SELECT IdT, Libelle
FROM hierarchie
WHERE IdT NOT IN (SELECT IdT FROM THEMQUEST)
```

- 1.15. Donnez par étudiant (numéro, nom) le nombre de questions du TP 1 et du TP2 réalisées. Formulez la requête avec imbrication(s) dans le FROM.

```
SELECT nom,prenom,COALESCE( (SELECT count(*)
                             FROM evaluation join question using(idq)
                             WHERE numtp=1 and numet=E.numet
                             GROUP BY numet),0) AS NbQTP1,
        COALESCE( (SELECT count(*)
                   FROM evaluation join question using(idq)
                   WHERE numtp=2 and numet=E.numet
                   GROUP BY numet),0) AS NbQTP2
FROM   etudiant E
ORDER BY 3 desc,4 desc
```

- 1.16. Affichez la moyenne des notes obtenues aux évaluations par TP (N°TP), ensuite par nom et prénom des étudiants (nom,prénom) et ensuite par N° de question (idq), vous réaliserez ces statistiques en une seule requête

```
select nom,prenom,numtp as "N° TP",idq "N° question",
        avg(EV.nbpoints) as "moyenne"
from   etudiant E join evaluation EV using(numet)
        join question Q using(idq)
group by grouping sets (numtp,(nom,prenom), idq)
order by numtp,nom;
```

- 1.17. Affichez la moyenne des notes obtenues aux évaluations par TP, nom, prénom et N° de question, ensuite par TP, nom, prénom et ensuite par TP ; vous réaliserez ces statistiques en une seule requête

```
select nom, prenom, numtp as "N° TP", idq "N° question",
       avg(EV.nbpoints) as "moyenne"
from etudiant E join evaluation EV using(numet)
               join question Q using(idq)
group by rollup (numtp, (nom, prenom), idq)
order by idq, numtp, nom;
```

- 1.18. Affichez la note totale obtenue par les étudiants dans chaque TP et le nombre de points max par TP

```
select nom, prenom, numtp as "N° TP", sum(EV.nbpoints) as "note",
       (select sum(nbpoints)
        from question where numtp = Q.numtp ) as "tot pts TP"
from etudiant E join evaluation EV using(numet)
               join question Q using(idq)
group by numtp, nom, prenom
order by numtp, nom;
```

- 1.19. Afficher les notes obtenues par les étudiants dans chaque TP ainsi que le rang de leur note par TP

```
with
  T1 (nom, prenom, Num_TP, note) as
  ( select nom, prenom, numtp, sum(EV.nbpoints)
    from etudiant E join evaluation EV using(numet)
                  join question Q using(idq)
    group by numtp, nom, prenom
    order by numtp, nom
  )
Select nom, prenom, Num_TP, note, DENSE_RANK() over(partition by Num_TP order by
note DESC) AS "Rang note"
from T1 ;
```

1. Indiquez la note maximale obtenue par étudiant, ensuite rendue par intervenant et pour terminer par matière. Vous n'écrirez qu'une seule requête pour cela.

```
SELECT Et.nom as "nomEtu", Et.prenom as "PrenomEtu",
       intitule as "Cours", I.nom as "nom Intervenant" ,
       I.prenom as "prenom Intervenant" ,
       MAX(note) as "Note max"
FROM Evaluation E JOIN Cours C ON E.idm = C.idm
      JOIN matiere M ON M.idm = C.idm
      JOIN intervenant I ON I.idi = C.idi
      JOIN etudiant Et ON Et.ide = E.ide
GROUP BY GROUPING SETS ((Et.ide, Et.nom , Et.prenom) ,
                        (M.idm, intitule) ,
                        ( I.nom , I.prenom, I.idi));
```

	ABC nomEtu ▼	ABC PrenomEtu ▼	ABC Cours ▼	ABC nom Intervenant ▼	ABC prenom Intervenant ▼	123 Note max ▼
1	[NULL]	[NULL]	PHP	[NULL]	[NULL]	18
2	[NULL]	[NULL]	BD1	[NULL]	[NULL]	19
3	[NULL]	[NULL]	C++	[NULL]	[NULL]	17
4	Lebeau	Remy	[NULL]	[NULL]	[NULL]	17
5	Tancarville	Laureline	[NULL]	[NULL]	[NULL]	17
6	Harkness	Jack	[NULL]	[NULL]	[NULL]	11
7	Connor	Sarah	[NULL]	[NULL]	[NULL]	18
8	Maximus	Magnus	[NULL]	[NULL]	[NULL]	19
9	Elric	Alphonse	[NULL]	[NULL]	[NULL]	14
10	Targaryen	Aegon	[NULL]	[NULL]	[NULL]	16
11	Underwood	Francis	[NULL]	[NULL]	[NULL]	11
12	Grimes	Rick	[NULL]	[NULL]	[NULL]	16
13	Yagami	Light	[NULL]	[NULL]	[NULL]	11
14	Batiatus	Quintus	[NULL]	[NULL]	[NULL]	18
15	White	Walter	[NULL]	[NULL]	[NULL]	18
16	[NULL]	[NULL]	[NULL]	Chauve	Nicolas	18
17	[NULL]	[NULL]	[NULL]	Martin	Michel	18
18	[NULL]	[NULL]	[NULL]	Restenst	Christian	19
19	[NULL]	[NULL]	[NULL]	Laka	Lotfi	17
20	[NULL]	[NULL]	[NULL]	Chettiti	Rosine	19
21	[NULL]	[NULL]	[NULL]	Codd	Ted	17
22	[NULL]	[NULL]	[NULL]	Mirandol	Serge	19
23	[NULL]	[NULL]	[NULL]	Carmigniac	Laurent	18

2. Dans quelle(s) salle(s) (nos et contenance) s'est / se sont déroulée(s) le plus de séances de cours ?

```
WITH R1 (nos, nbr) AS
(
    select distinct Nos, count(*) over (partition by Nos) as nb
    from Cours
    order by nb desc
)
select nos, contenance
from R1 join salle using(nos)
where nbr=(select max(nbr) from R1)
```

ABC nos ▼	123 contenance ▼
E103	29

3. Donnez le classement des cinq meilleurs étudiants de la promo en affichant également leurs noms et moyennes obtenues.

```
WITH Tmp (ide, moy, classement) AS
(
    SELECT ide, AVG(note)::decimal(6,3),
           DENSE_RANK() OVER (ORDER BY AVG(note) DESC)
    FROM Evaluation
    GROUP BY ide
)
SELECT nom, moy, classement
FROM Tmp T INNER JOIN Etudiant E ON T.ide = E.ide
WHERE classement <= 5
ORDER BY classement ASC;
```

	ABC nom	123 moy	123 classement
1	Lebeau	16	1
2	Connor	16	1
3	Maximus	15,333	2
4	Grimes	14,667	3
5	White	14,333	4
6	Targaryen	14,333	4
7	Batiatus	14	5

4. Quels sont les noms et statuts des intervenants qui ont assuré toutes les séances de cours recensées de chaque matière ?

Par exemple si pour chaque matière il y a :

BD1 : 3 séances, C++ : 3 séances, PHP : 4 séances de préviews

Alors on cherche les intervenants qui sont intervenus :
3 fois en BD1 , 3 fois en C++ et 4 fois en PHP

```
WITH
T1 (idm, nbs) AS
(
    SELECT idm, MAX(nums) AS nbs
    FROM Cours
    GROUP BY idm
)
SELECT C.idi ,nom AS intervenant, statut
FROM Intervenant I JOIN Cours C ON C.idi = I.idi
GROUP BY C.idi,nom, statut
HAVING COUNT(DISTINCT idM) = (SELECT COUNT(DISTINCT idM) FROM
Cours)
AND COUNT(nums) = (SELECT SUM(nbs) FROM T1);
```

123 idi	ABC intervenant	ABC statut
2	Miranda	P

5. Pour chaque salle qui accueille des cours et de contenance supérieure ou égale à 20, indiquez le nombre de cours par salle, ensuite par matière (idm), ensuite par intervenant et ensuite le nombre total de cours. Vous n'écrirez qu'une seule requête pour cela.

```
SELECT S.nos,idm,idi, COUNT(C.nos) AS nb
FROM Salle S JOIN Cours C ON S.nos = C.nos
WHERE Contenance >= 20
GROUP BY grouping sets (S.nos,idm, idi,())
order by 1, 2, 3;
```

ABC nos	123 idm	123 idi	123 nb
E102	[NULL]	[NULL]	4
E103	[NULL]	[NULL]	8
E105	[NULL]	[NULL]	6
E106	[NULL]	[NULL]	3
E107	[NULL]	[NULL]	2
[NULL]	11	[NULL]	6
[NULL]	12	[NULL]	6
[NULL]	13	[NULL]	11
[NULL]	[NULL]	1	1
[NULL]	[NULL]	2	10
[NULL]	[NULL]	3	2
[NULL]	[NULL]	4	2
[NULL]	[NULL]	5	2
[NULL]	[NULL]	6	3
[NULL]	[NULL]	7	2
[NULL]	[NULL]	8	1
[NULL]	[NULL]	[NULL]	23

6. Afficher la matière, le nom et prénom de l'étudiant, son groupe, la note obtenue, son rang (sans trou) et la note maximum pour chaque matière (intitule)

```
select intitule, nom, prenom, groupe, note,
       dense_rank() over w as "classement",
       first_value(note) over w as "note Max"
from etudiant e join evaluation e2 using(ide)
               join matiere m using(idm)
window w as (partition by intitule order by note desc )
order by intitule, "classement", nom
```

	ABC intitule	ABC nom	ABC prenom	ABC groupe	123 note	123 classement	123 note Max
1	BD1	Maximus	Magnus	G3	19	1	19
2	BD1	Batius	Quintus	G2	18	2	19
3	BD1	Connor	Sarah	G2	17	3	19
4	BD1	Lebeau	Remy	G2	17	3	19
5	BD1	Grimes	Rick	G1	15	4	19
6	BD1	White	Walter	G1	14	5	19
7	BD1	Targaryen	Aegon	G3	13	6	19
8	BD1	Elric	Alphonse	G1	12	7	19
9	BD1	Harkness	Jack	G2	11	8	19
10	BD1	Tancarville	Laureline	G3	11	8	19
11	BD1	Yagami	Light	G1	10	9	19
12	BD1	Underwood	Francis	G3	9	10	19
13	C++	Tancarville	Laureline	G3	17	1	17
14	C++	Grimes	Rick	G1	16	2	17
15	C++	Targaryen	Aegon	G3	16	2	17
16	C++	Lebeau	Remy	G2	15	3	17
17	C++	Maximus	Magnus	G3	15	3	17

..... 36 lignes

7. Afficher l'intitulé des matières, le groupe, le nombre de séances prévues (nbs) et le nombre de séances effectives par matière et par groupe

```
SELECT distinct intitule, groupe, nbs as nbs_prevu,
COUNT(*) over (partition by groupe, idm ) as nbs_realise
FROM matiere M join cours C using(idm)
order by 1,2;
```

intitule	groupe	nbs_prevu	nbs_realise
BD1	G1	3	1
BD1	G2	3	2
BD1	G3	3	3
C++	G1	3	2
C++	G2	3	3
C++	G3	3	1
PHP	G1	4	4
PHP	G2	4	4
PHP	G3	4	4

8. Donnez les matières pour lesquelles toutes les séances ont été assuré, et ce pour chaque groupe.

Vous devez compter le nombre de séance prévu par groupe, en faire la somme et la comparer avec la somme du nombre de séances réalisé par groupe!

```
with R1 (intitule, idm ,groupe, nbs_prevue,nbs_realise ) AS
(
    SELECT distinct intitule,idm , groupe, nbs,
    COUNT(*) over (partition by groupe, idm ) as nbsr
    FROM matiere M join cours C using(idm)
),
R2 (intitule,idm,tot_realise,tot_prevu) as
(
    SELECT intitule,idm,sum(nbs_realise) as tot_realise,
    sum(nbs_prevue) as tot_prevu
    FROM R1
    GROUP BY intitule,idm
    ORDER BY 1,2
)
SELECT * FROM R2
WHERE tot_prevu=tot_realise;
```

intitule	idm	tot_realise	tot_prevu
PHP	13	12	12

9. Pour chaque matière, afficher la matière, le nom, prénom et la note obtenue de l'étudiant qui a la meilleure note dans la matière !

```

with R1 ( idm, intitule,ide, nom, prenom, groupe, note,rang, noteMax)
AS
(
    select idm,intitule,ide, nom, prenom, groupe, note,
    dense_rank() over w ,
    first_value(note) over w
    from etudiant e join evaluation e2 using(ide)
    join matiere m using(idm)
    window w as (partition by intitule order by note desc )
)
select distinct R1.intitule, R1.nom, R1.prenom, R1.note
from evaluation e3 join matiere m2 using(idm)
join R1 using(idm)
where m2.idm=R1.idm
and R1.note =notemax
order by R1.intitule

```

ABC intitule	ABC nom	ABC prenom	123 note
BD1	Maximus	Magnus	19
C++	Tancarville	Laureline	17
PHP	Connor	Sarah	18
PHP	White	Walter	18

10. Afficher la matière, le nom et prénom de l'étudiant et sa note obtenue dans chaque matière, la note minimum par matière, la moyenne des notes par matière !

```

SELECT m.intitule, nom, prenom, note,
    AVG(note) OVER (partition BY m.intitule, m.idm ) as moyenne,
    MIN(note) OVER (partition BY m.intitule, m.idm ) as note_min
FROM evaluation e join matiere m on e.idm =m.idm
join etudiant e2 on e2.ide=e.ide

```

ABC intitule	ABC nom	ABC prenom	123 note	123 moyenne	123 note_min
BD1	Elric	Alphonse	12	13,8333333333	9
BD1	Yagami	Light	10	13,8333333333	9
BD1	White	Walter	14	13,8333333333	9
BD1	Grimes	Rick	15	13,8333333333	9
BD1	Harkness	Jack	11	13,8333333333	9
BD1	Batiatus	Quintus	18	13,8333333333	9
BD1	Lebeau	Remy	17	13,8333333333	9
BD1	Connor	Sarah	17	13,8333333333	9
BD1	Tancarville	Laureline	11	13,8333333333	9
BD1	Underwood	Francis	9	13,8333333333	9
BD1	Targaryen	Aegon	13	13,8333333333	9
BD1	Maximus	Magnus	19	13,8333333333	9
C++	Elric	Alphonse	14	13,1666666667	9
C++	Yagami	Light	11	13,1666666667	9
C++	White	Walter	11	13,1666666667	9
C++	Grimes	Rick	16	13,1666666667	9
C++	Harkness	Jack	10	13,1666666667	9
C++	Batiatus	Quintus	9	13,1666666667	9

11. Afficher la matière qui a la meilleure moyenne. Donner 2 formulations différentes

```
SELECT m.intitule
FROM evaluation e join matiere m on e.idm =m.idm
group by intitule, m.idm
having avg(note) >=all (select avg(note)
                        FROM evaluation e join matiere m on e.idm =m.idm
                        group by intitule, m.idm )
```

ou

```
with
R1 (mat, moy) AS
(
    SELECT m.intitule,avg(note)
    FROM evaluation e join matiere m on e.idm =m.idm
    group by intitule, m.idm
),
R2 (maxi) as
(
    select max(moy) from R1
)
select mat
from R1
where moy = (select maxi from R2)
```

ABC mat	↑↓
BD1	

12. Quel intervenant (nom et prénom) a assuré le moins de cours ? **Donner 2 formulations différentes**

```
select nom, prenom
from cours c join intervenant i on i.idi=c.idi
group by i.idi, nom, prenom
order by count(*)
limit 1
```

ou

nom	prenom
Codd	Ted

```
select nom, prenom
from cours c join intervenant i on i.idi=c.idi
group by i.idi, nom, prenom
having count(*) <= ALL(select count(*)
                        from cours
                        group by idi)
```

ou

```
with
R1 (nom, prenom, nbi) as
(
    select nom, prenom, count(*)
    from cours c join intervenant i on i.idi=c.idi
    group by i.idi, nom, prenom
),
R2 (mini) as
(
    select min(nbi) from R1
)
select nom, prenom
from R1
where nbi=(select mini from R2)
```

1. Donner le nombre de note par étudiant, puis par société et enfin total, **en une seule requête**

On veut les colonnes suivantes :

nom étudiant nom société nb_note

```
SELECT E.nom as "nom étudiant", S.nom as "nom société", count(*) AS nb_note
FROM convention C join etudiant E using(id)
      join societe S using(ids)
GROUP BY GROUPING SETS(E.nom, S.nom, ());
```

2. Donnez le classement des deux meilleurs étudiants en stage (nom, prenom) de la promo en affichant également leurs noms et moyennes obtenues.

```
WITH Tmp (ide, moy, classement) AS
(
    SELECT ide, AVG(note), RANK() OVER (ORDER BY AVG(note) DESC)
    FROM convention
    GROUP BY ide
)
SELECT nom, prenom, moy, classement
FROM Tmp T INNER JOIN Etudiant E
    ON T.ide = E.ide
WHERE classement <= 2
ORDER BY classement ASC;
```

3. Afficher la moyenne des salaires par société (nom). On veut voir sur chaque ligne le nom de la société, le nom, prénom du personnel, le salaire du personnel, la moyenne des salaires par société (ids) et enfin le salaire qui est juste en dessous le salaire le plus élevé par société

On veut les colonnes suivantes :

nom société nom personnel prenom salaire moyenne 2eme salaire le plus haut

```
SELECT S.nom AS "nom société", P.nom as "nom personnel", prenom, salaire,
      AVG(salaire)
      OVER (PARTITION BY ids ORDER BY ids DESC) as moyenne,
      nth_value(salaire, 2) OVER (PARTITION BY ids ORDER BY salaire DESC
      RANGE BETWEEN UNBOUNDED PRECEDING and UNBOUNDED FOLLOWING
) as "2eme salaire le plus haut"
FROM personnel P join societe S using(ids)
order by 1, 4 desc
```

4. Donnez le rang des salaires (1 pour le plus élevé et N pour le moins élevé) des personnels par société (ids) en affichant également leurs noms et leur salaire

On veut les colonnes suivantes :

ids nom prenom salaire rang des salaires

```
SELECT ids, nom, prenom, salaire,
      rank()
      OVER (PARTITION BY ids ORDER BY salaire DESC) as "rang des salaires"
FROM personnel;
```

5. Donnez le salaire moyen par société, le salaire max et min par société, des personnels en affichant également leur nom, prénom, leur salaire et le nom de la société dans laquelle ils travaillent
On veut les colonnes suivantes :

nom société nom personnel prenom salaire Moyenne des salaires Maximum des salaires Minimum des salaires

```
SELECT S.nom as "nom société", P.nom as "nom personnel", prenom, salaire,
AVG(salaire) OVER (PARTITION BY S.ids) as "Moyenne des salaires",
MAX(salaire) OVER (PARTITION BY S.ids) as "Maximum des salaires",
MIN(salaire) OVER (PARTITION BY S.ids) as "Minimum des salaires"
FROM personnel P join societe S ON S.ids=P.ids
order by S.nom
```

6. Pour chaque société (nom) donnez les employés (nom et prénom) qui sont directement sous les ordres du directeur (ceux qui n'ont pas de père)
On veut les colonnes suivantes :

nom société nom personnel prenom

```
SELECT S.nom as "nom société", P.nom, prenom
FROM personnel P JOIN societe S ON P.ids=S.ids
WHERE idppere IN (SELECT idp FROM personnel WHERE idppere IS NULL)
```

7. Pour chaque société (nom) donnez les employés (nom et prénom) qui n'ont personne sous leurs ordres (les feuilles de l'arbre)
On veut les colonnes suivantes :

nom société nom personnel prenom

```
SELECT S.nom as "nom société", P.nom prenom
FROM personnel P JOIN societe S ON P.ids=S.ids
WHERE idp NOT IN (SELECT idppere FROM personnel
                  WHERE idppere IS NOT NULL)
```

8. Donnez toute la hiérarchie descendante des employés (nom et prénom), en partant du big boss de la société Almacom

```
WITH RECURSIVE hierarchie (nom, prenom, idp, idppere) AS
(
  SELECT P.nom, prenom, idp, idppere
  FROM personnel P JOIN societe S ON P.ids=S.ids
  WHERE idppere IS NULL
  AND S.nom='Almacom'

  UNION ALL

  SELECT P.nom, P.prenom, P.idp, P.idppere
  FROM personnel P JOIN hierarchie H ON H.idp=P.idppere
)
SELECT nom, prenom FROM hierarchie ;
```

9. Donnez toute la hiérarchie descendante des employés (nom et prénom), en partant du big boss de toutes les sociétés (nom), trié par société et par hiérarchie décroissante

```
WITH RECURSIVE hierarchie (societe,nom,prenom,idp,idppere) AS
(
    SELECT S.nom,P.nom, prenom, idp,idppere
    FROM personnel P JOIN societe S ON P.ids=S.ids
    WHERE idppere IS NULL

    UNION ALL

    SELECT H.societe,P.nom, P.prenom, P.idp,P.idppere
    FROM personnel P JOIN hierarchie H ON H.idp=P.idppere
)
SELECT societe,nom,prenom from hierarchie order by societe,idp ;
```

10. Donnez tous les employés (nom et prénom), qui sont au-dessus de Martin Anne, elle ne doit pas apparaitre dans le résultat.

```
WITH RECURSIVE hierarchie (nom,prenom,idp,idppere) AS
(
    SELECT nom, prenom, idp,idppere
    FROM personnel
    WHERE nom='Martin' AND prenom ='Anne'

    UNION ALL

    SELECT P.nom, P.prenom, P.idp,P.idppere
    FROM personnel P JOIN hierarchie H ON H.idppere=P.idp
)
SELECT nom,prenom from hierarchie
WHERE nom!='Martin' AND prenom !='Anne';
```

11. Donnez tous les personnels (nom et prénom), qui n'ont encadré un stage, on exclura le big boss.

```
SELECT S.nom,P.nom, prenom
FROM personnel P JOIN societe S ON P.ids=S.ids
WHERE idp NOT IN (select idp from convention)
AND idppere IS NOT NULL
```